

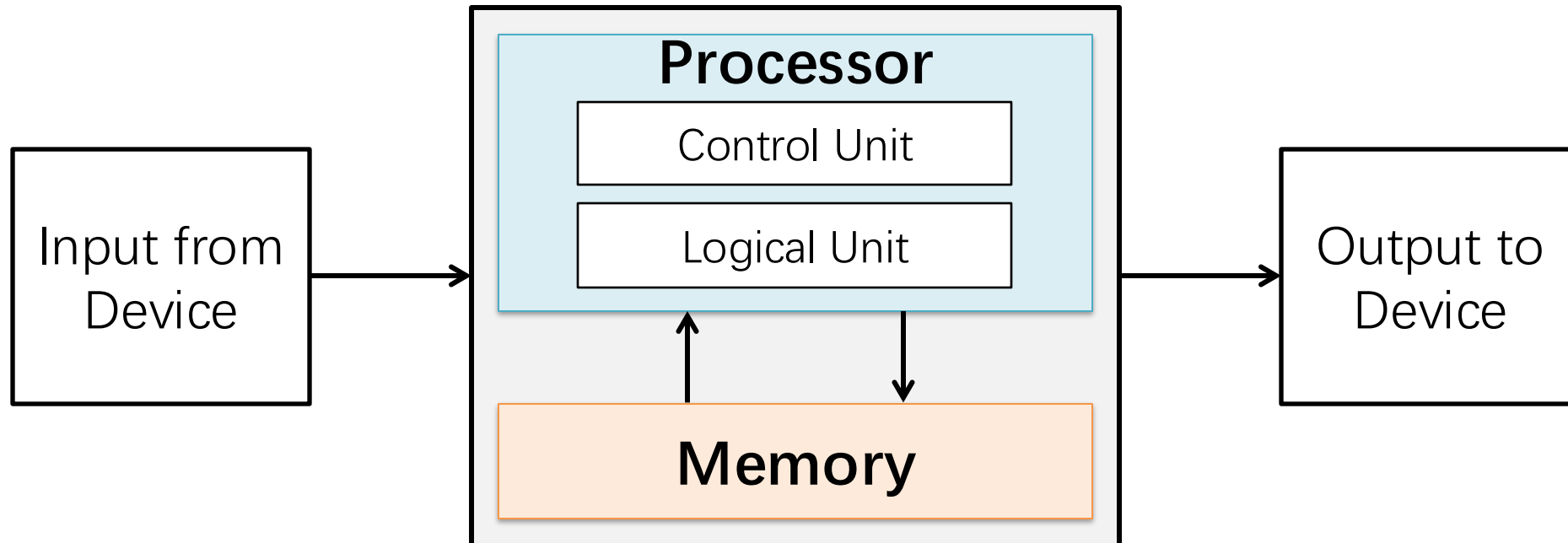
LECTURE 12
Memory Management (I)

Xuhao Chen

March 17, 2025



Von-Neumann Architecture



Memory Management

- It is about **efficiently allocating, using, and reclaiming memory** to improve **performance, scalability**, and security.
- We'll talk about
 - ▣ Virtual Memory & Address Translation
 - ▣ Memory Allocation Strategies
 - ▣ Garbage Collection (Automatic Memory Management)
 - ▣ Memory Hierarchy and Organization

Outline

- Virtual Memory & Address Translation
- Memory Allocation Strategies
 - ▣ Stacks
 - ▣ Heaps
 - ◆ Metrics to evaluate Allocators
 - ◆ Fixed-size Heap Allocation
 - ◆ Variable-size Heap Allocation
 - ◆ Parallel Heap Allocation
- Garbage Collection
- Memory Hierarchy and Organization

VIRTUAL MEMORY SYSTEM

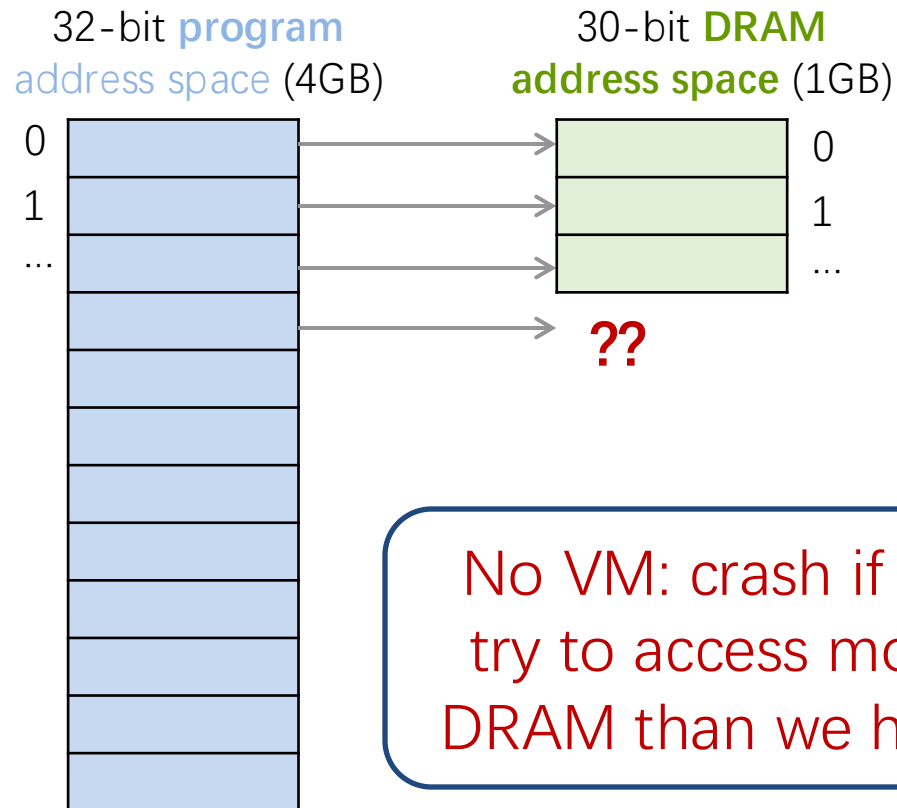


Virtual Memory System

All problems in computer science can be solved by **another level of indirection** — Butler Lampson

Without Virtual Memory

Program Address = DRAM address



No VM: crash if we
try to access more
DRAM than we have

With Virtual Memory

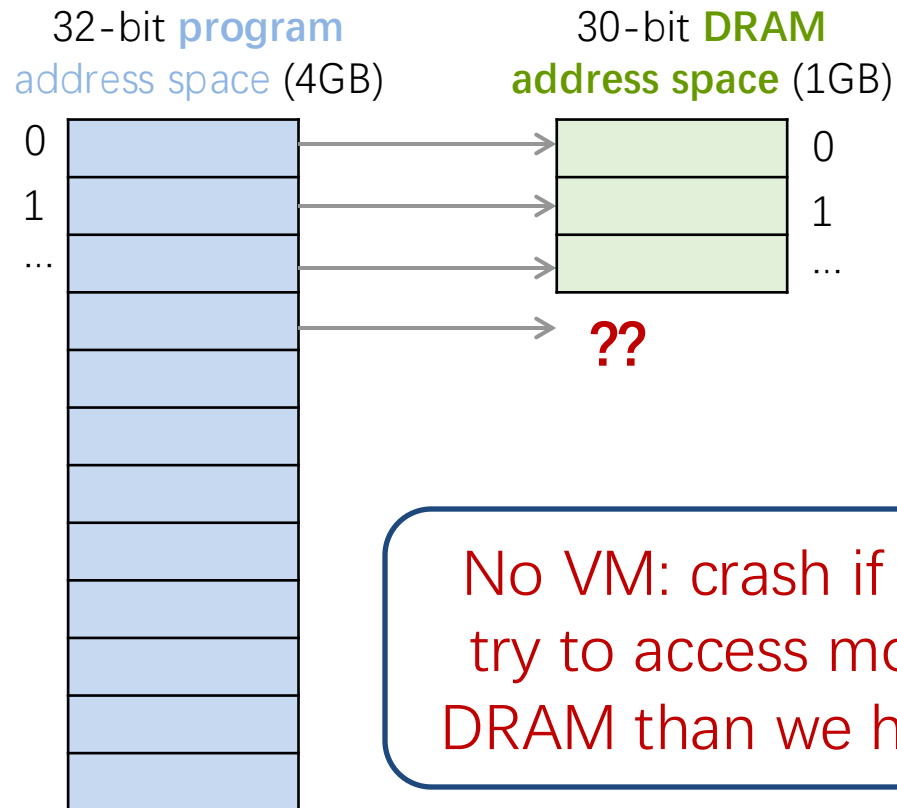
Program Address **Maps to** DRAM address

Virtual Memory System

All problems in computer science can be solved by **another level of indirection** — Butler Lampson

Without Virtual Memory

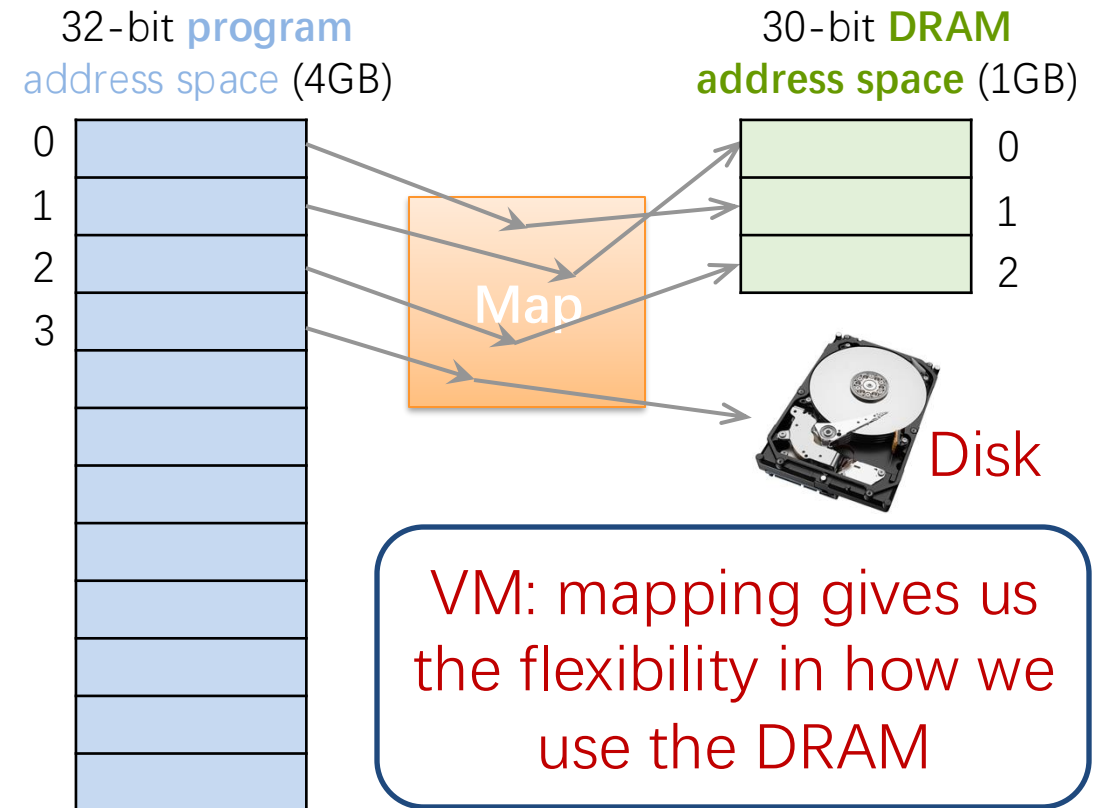
Program Address = DRAM address



No VM: crash if we try to access more DRAM than we have

With Virtual Memory

Program Address Maps to DRAM address

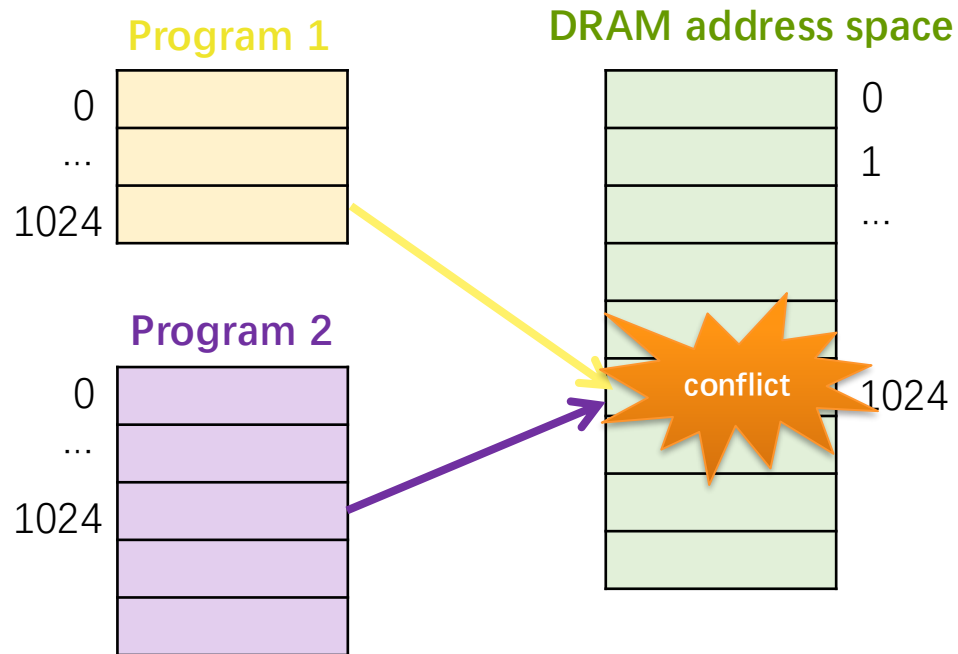


VM: mapping gives us the flexibility in how we use the DRAM

Virtual Memory System

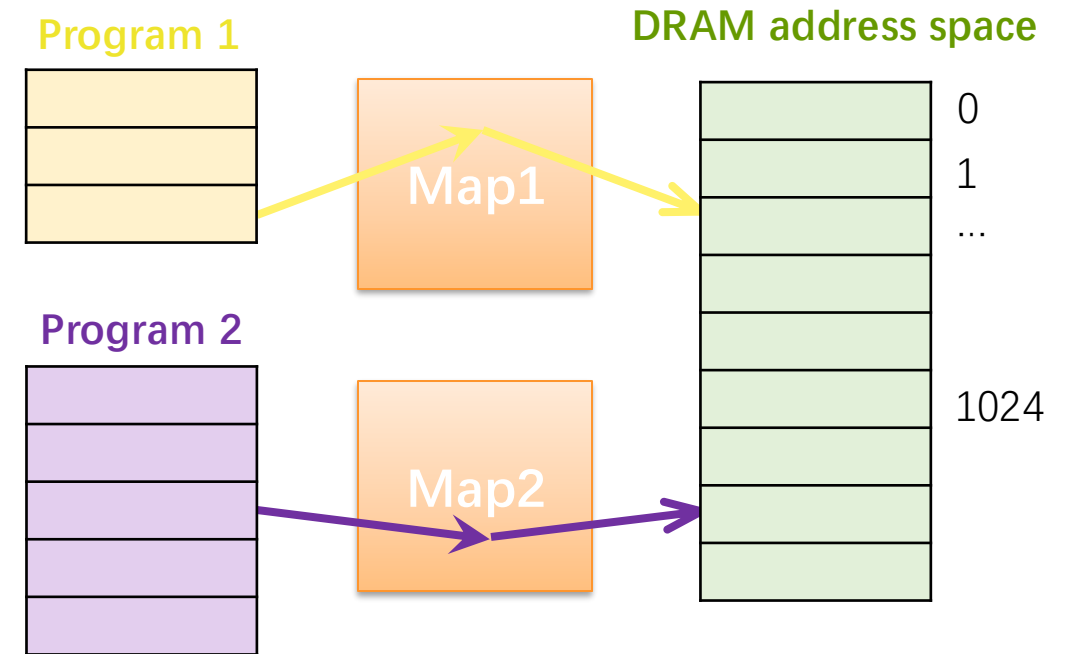
Without Virtual Memory

Program Address = DRAM address



With Virtual Memory

Program Address Maps to DRAM address



Program 1 stores your bank balance at address 1024
Program 2 stores your video game score at address 1024

Virtual Memory: abstract storage resources

- **Protection & Privacy**

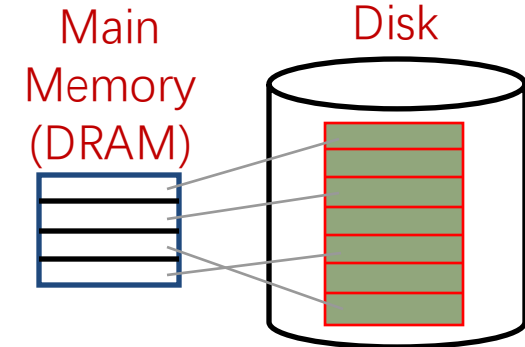
- Each process has a **private** address space



An **illusion** of a large, private, uniform store

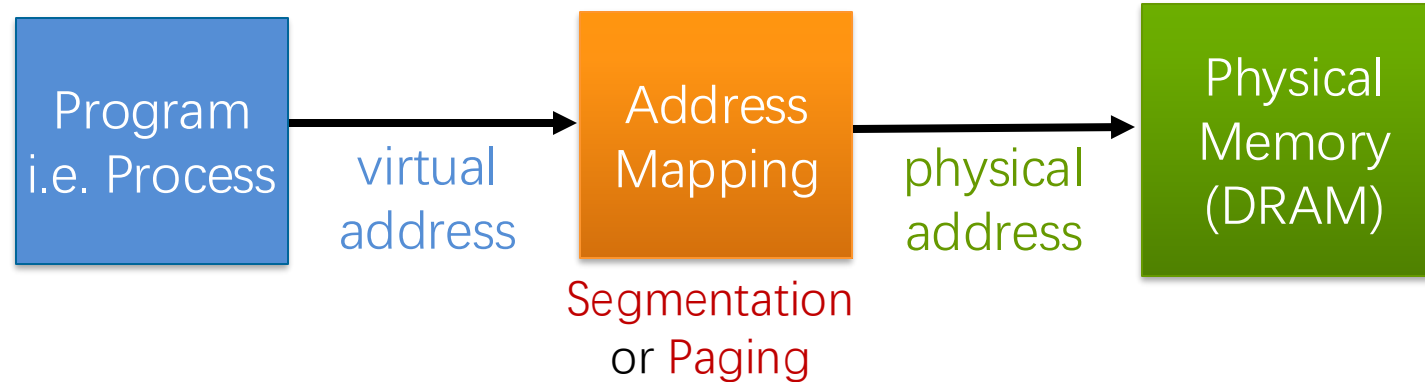
- **Demand Paging**

- Use DRAM as a **cache**/buffer of disk
- Enables running programs **larger** than DRAM
- DRAM holds a **portion** of the process's data



The price of Virtual Memory is
address translation on each memory reference

Virtual Address vs. Physical Address



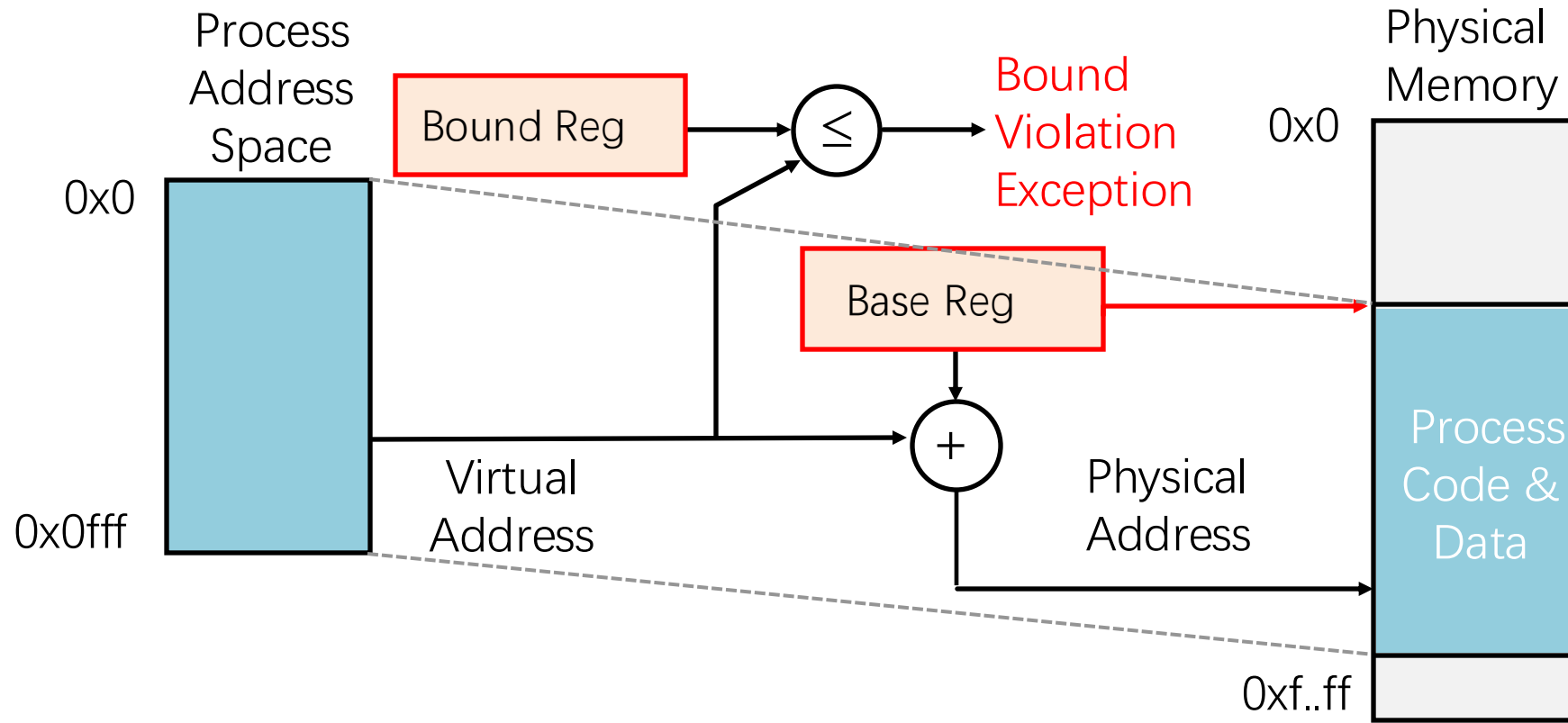
- **Virtual address**

- ▣ Address generated by the program (i.e., process)
- ▣ Specific to the process's **private** address space

- **Physical address**

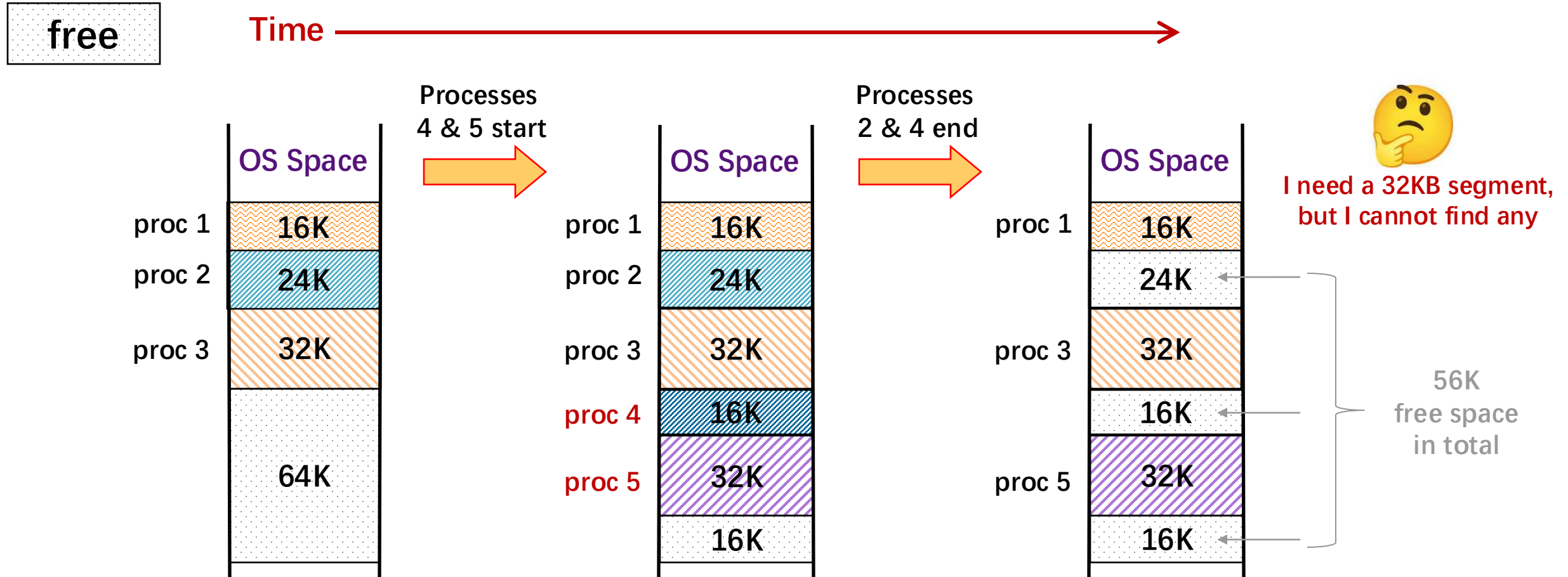
- ▣ Address used to access physical (hardware) memory
- ▣ **OS** manages address mapping

Segmentation: *Base-and-Bound* Address Translation



- Each program's data is allocated in a **contiguous** segment of physical memory
- **Physical Address = Virtual Address + Segment Base**
- Bound register provides safety and isolation
- Base and Bound registers are **NOT accessible** by user programs (only in supervisor mode)

Memory Fragmentation



- As processes start and end, storage is “**fragmented**”
- Segments have to be **moved** around to compact the free space

Paged Memory Systems

- Divide physical memory in *fixed-size blocks* called **pages**

- ▣ Typical page size: 4KB (2^{12} bytes)

- Each virtual address is a pair:

<virtual page number, offset>

32-p bits

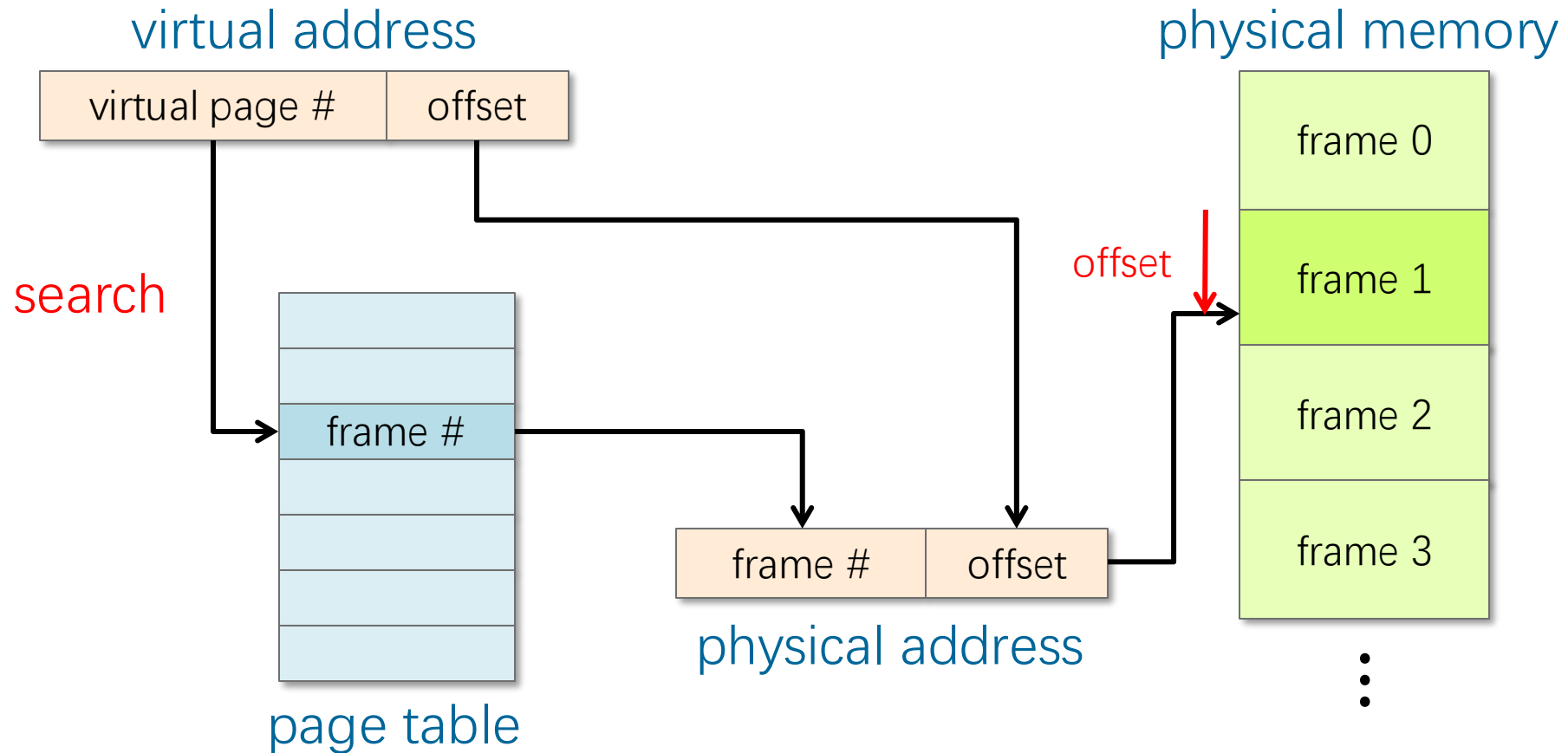
p bits

(p=12 for 4K page)

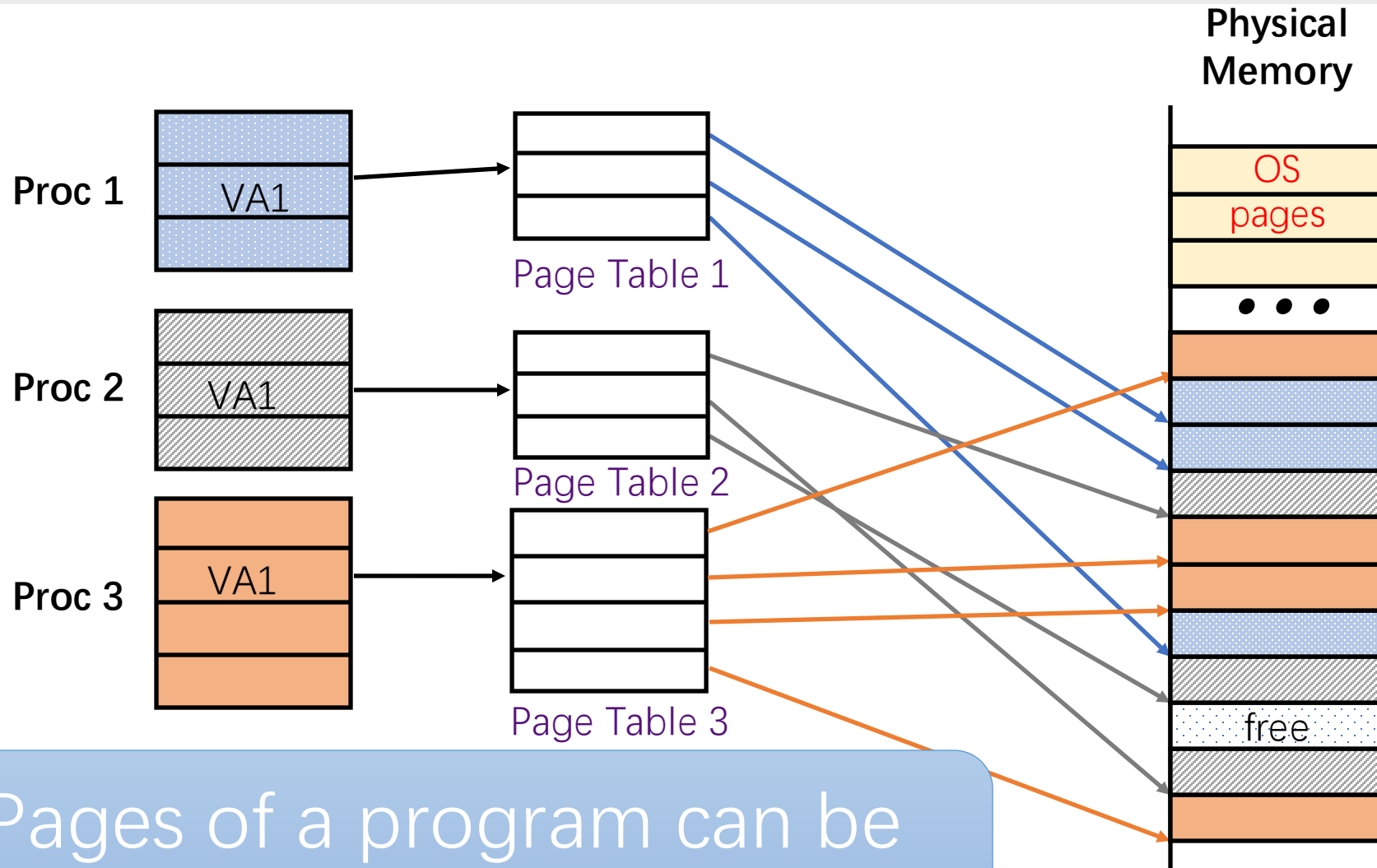
- **Page Table**: translate from **virtual page** # to **physical page** #

Pages of a program can be stored **non-contiguously**

Address Translation



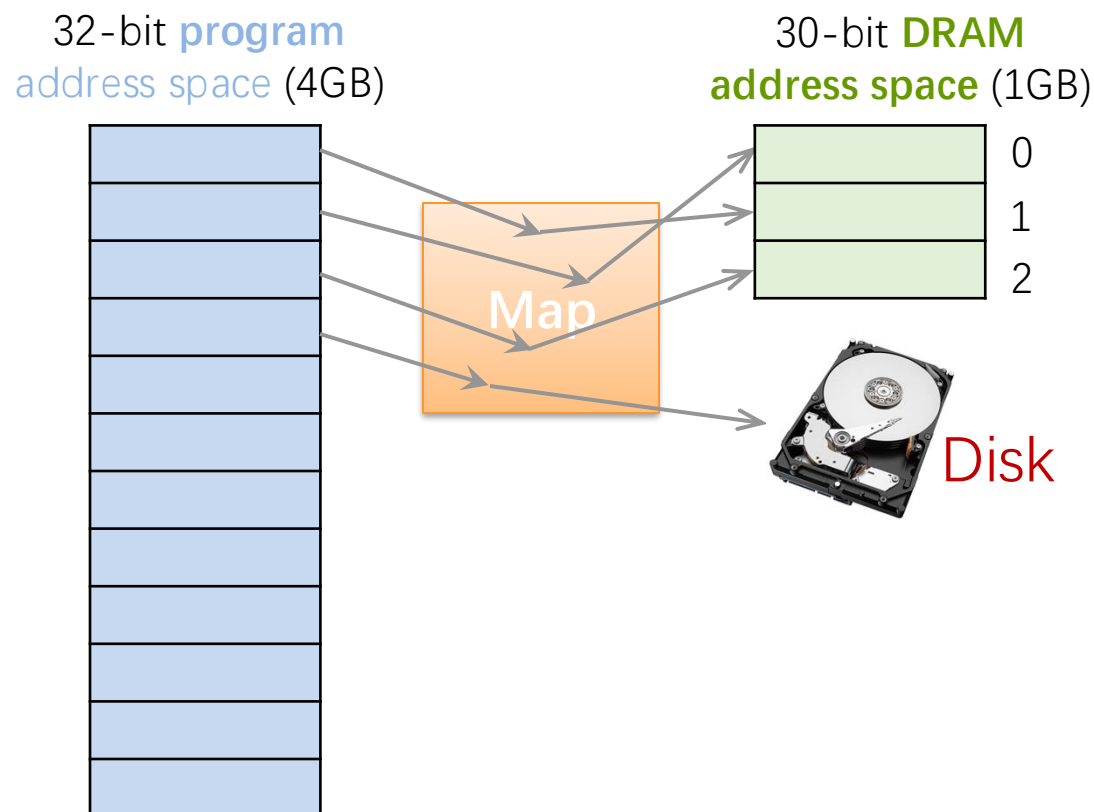
Private Address Space per Process



Pages of a program can be stored **non-contiguously**

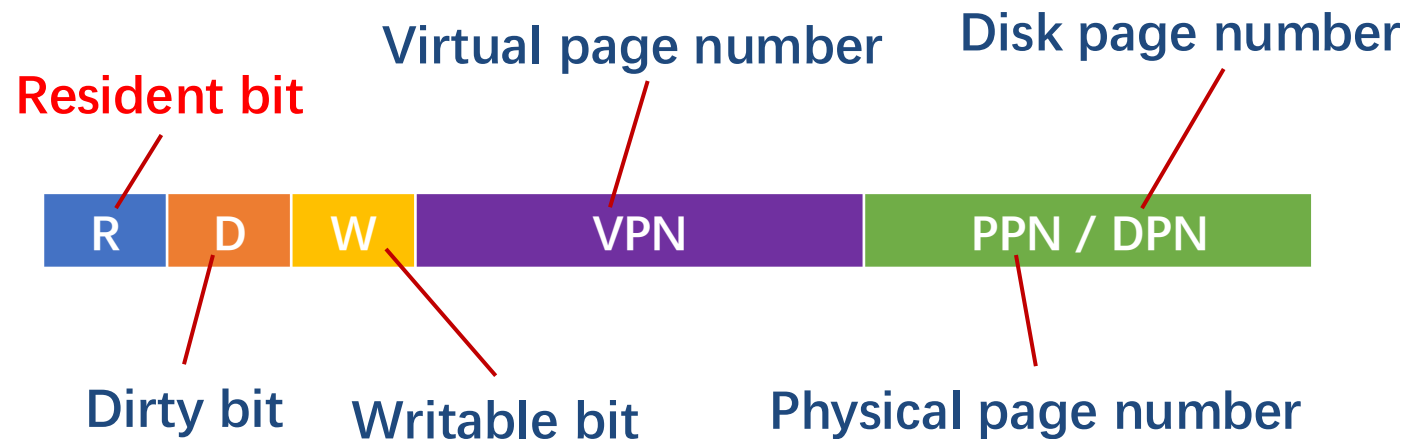
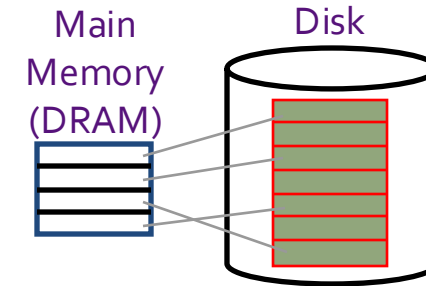
Questions

- What if a page does not reside in physical memory
 - ▣ Not all the pages can fit in DRAM



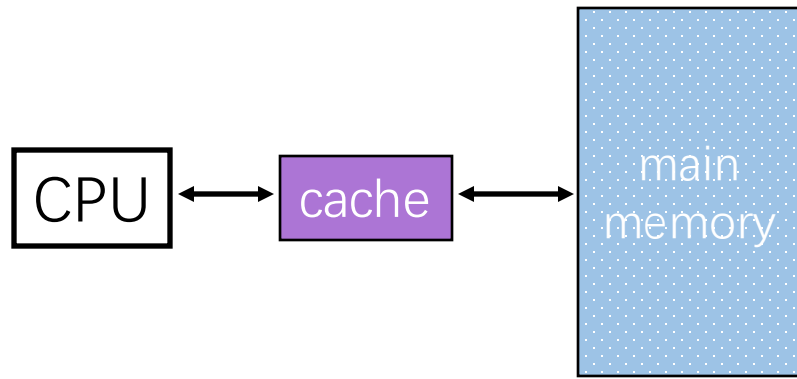
Demand Paging: using DRAM as a cache of disk

- DRAM is backed up by *swap space* on disk
- Page Table Entry (PTE) contains:



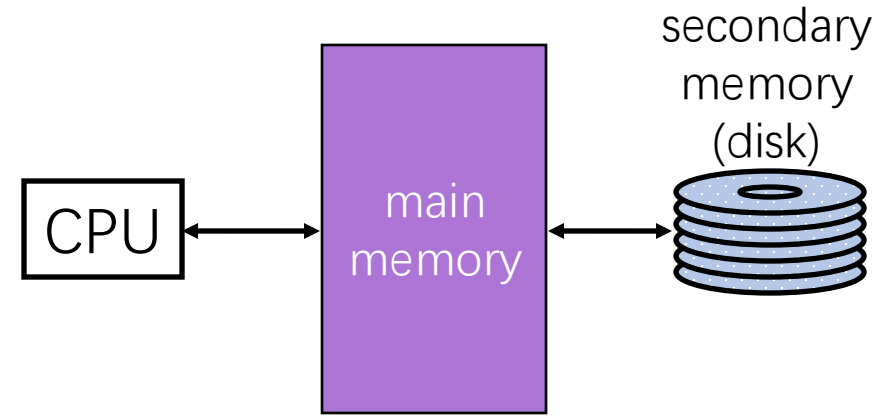
If $R=1$, PPN (physical page number) for a memory-resident page
If $R=0$, DPN (disk page number) for a page on the disk

Caching vs. Demand Paging



Caching

cache entry
cache block (~32 bytes)
cache miss rate (1% to 20%)
cache hit (~1 cycle)
cache miss (~100 cycles)
a miss is handled in *hardware*



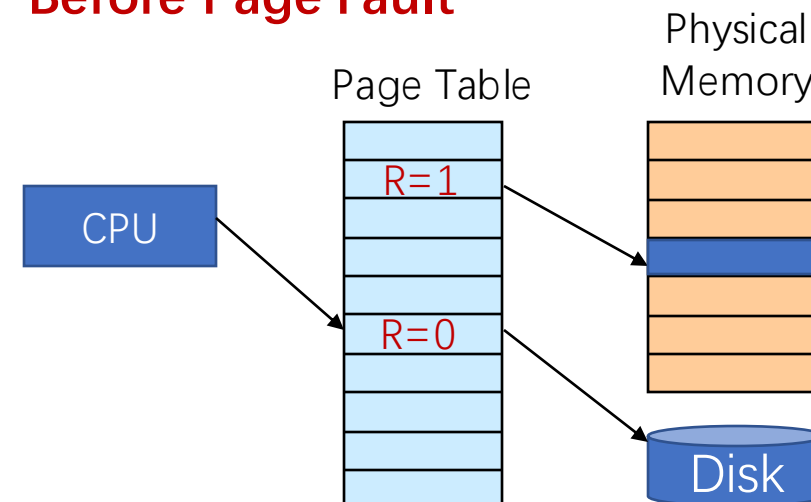
Demand paging

page frame
page (~4K bytes)
page miss rate (<0.001%)
page hit (~100 cycles)
page miss (~5M cycles)
a miss is handled mostly in *software*

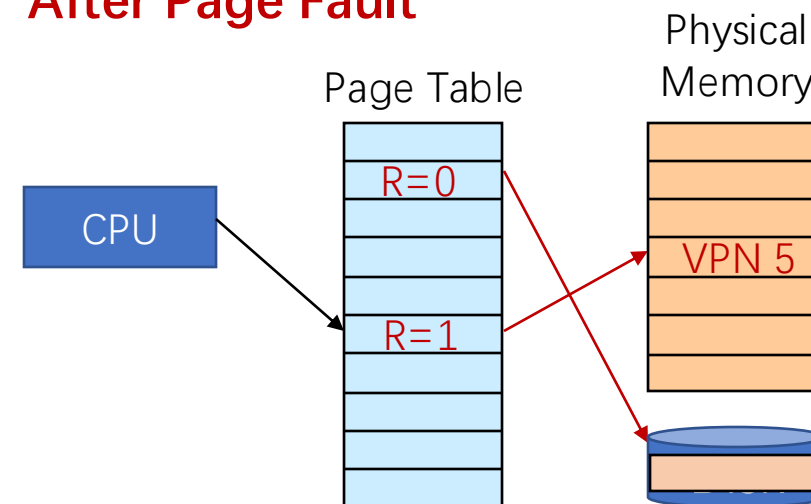
Page Faults

- An access to a page with **R=0** causes a page fault exception
- OS page fault handler is invoked:
 - ▣ Choose a page to replace, write it back if dirty. Mark page as non-resident
 - ▣ Read page from disk into available physical page
 - ▣ Update page table to show new page is resident
 - ▣ Return control to program, which re-executes memory access
 - ▣ a **page fault** occurs ~1,000,000 cycles

Before Page Fault



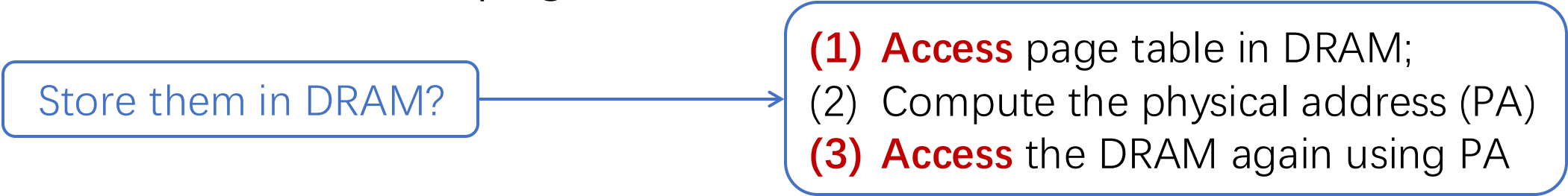
After Page Fault



Questions

- What if a page does not reside in physical memory
 - ▣ Not all the pages can be fit in DRAM
 - ▣ Demand Paging & Page Fault
- Where to store the page tables?

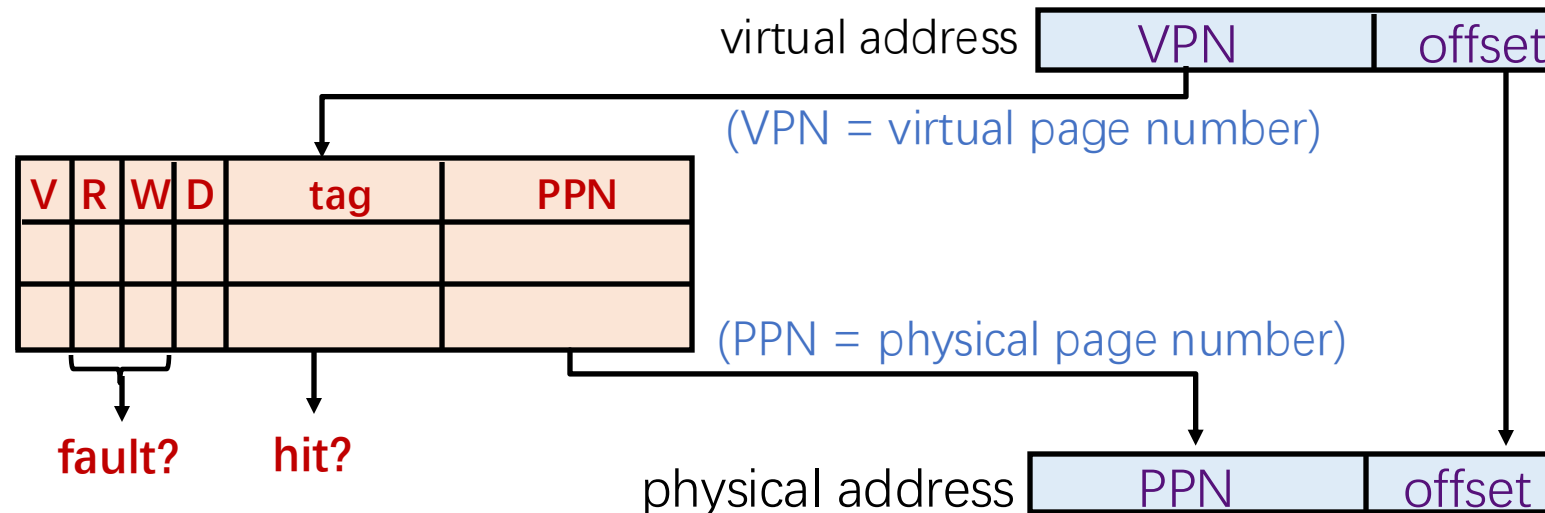
Store them in DRAM?

- 
- (1) **Access** page table in DRAM;
 - (2) Compute the physical address (PA)
 - (3) **Access** the DRAM again using PA

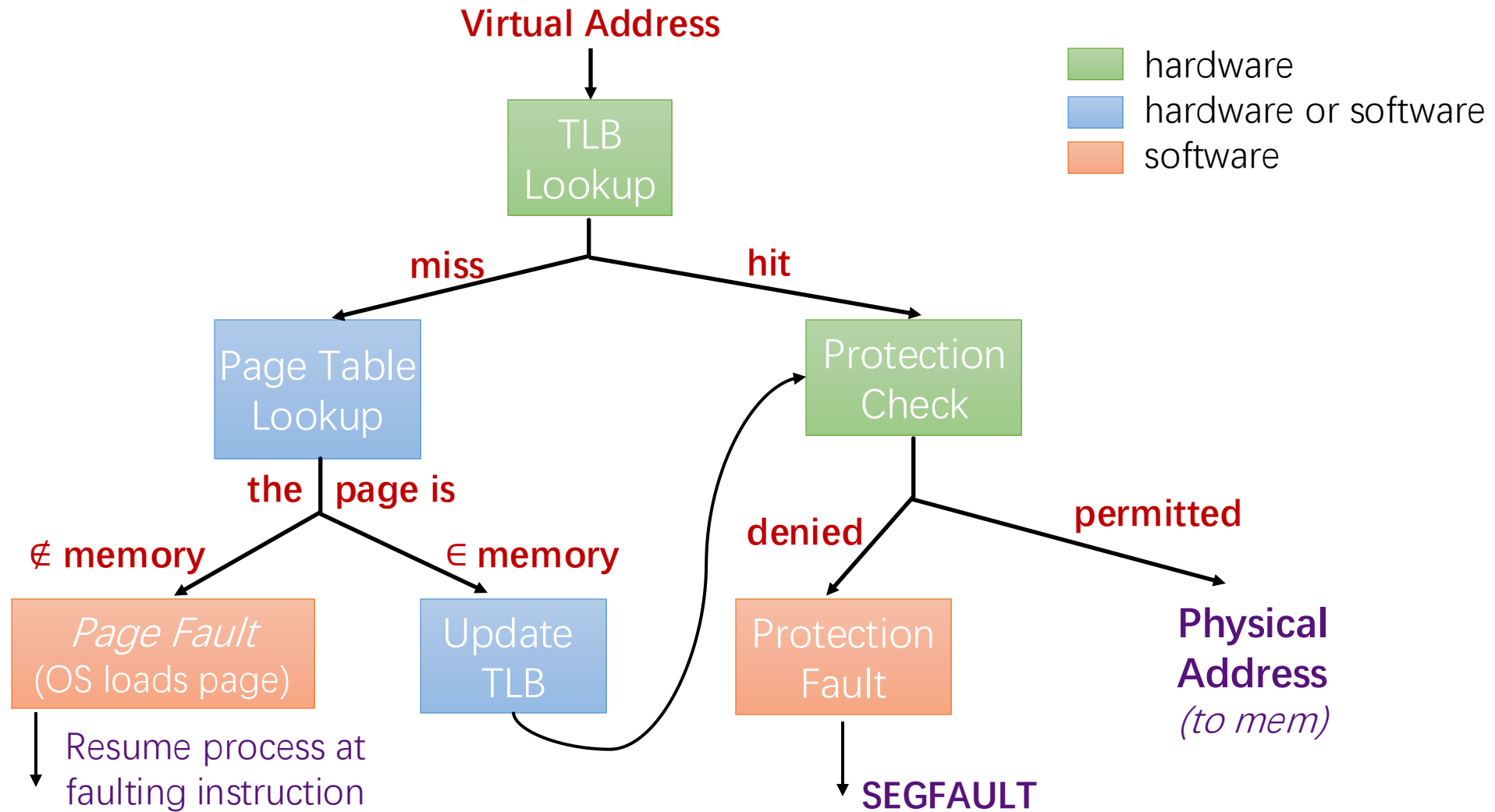
Accessing **one** word data
requires **two** DRAM accesses!!!

Translation Lookaside Buffer (TLB)

- **Problem:** Address translation is very expensive!
 - ▣ Each reference requires accessing page table
- **Solution:** Cache translations in TLB
 - ▣ TLB hit → *Single-cycle translation*
 - ▣ TLB miss → *Access page table to refill TLB*



Address Translation: *Putting it all together*



Outline

- Virtual Memory System
- Stacks
- Heaps
 - Metrics to evaluate Allocators
 - Fixed-size Heap Allocation
 - Variable-size Heap Allocation
 - Parallel Heap Allocation

STACKS



Stack Allocation

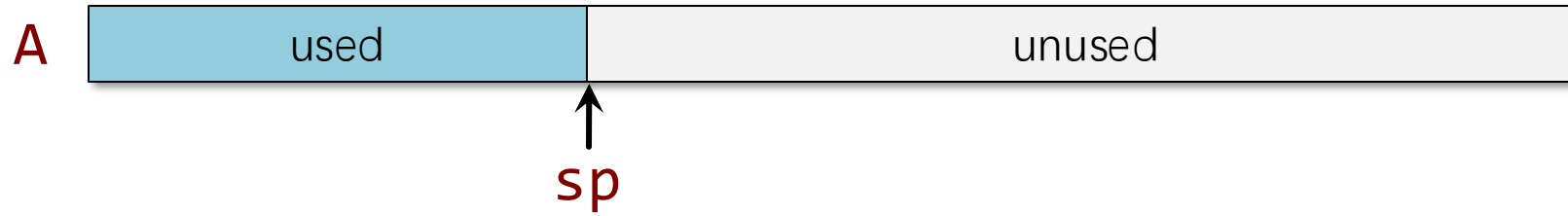
Stack discipline

- **LIFO** (last in, first out)
- The object that was most recently allocated (**pushed**) is the next to be freed (**popped**)

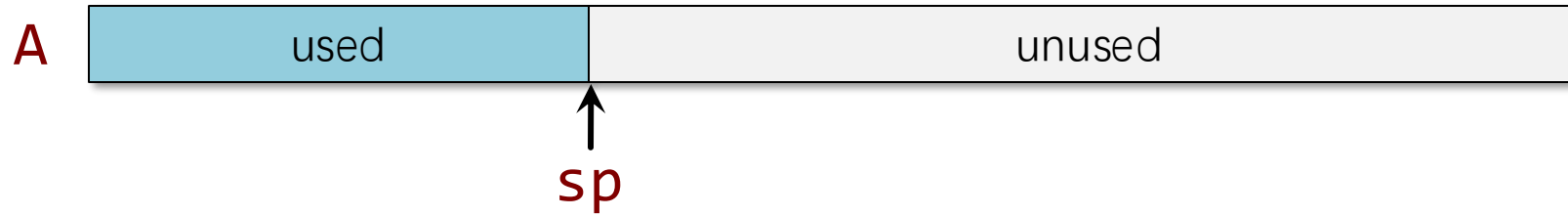


A cafeteria plate dispenser obeys a stack discipline.

Array and Pointer



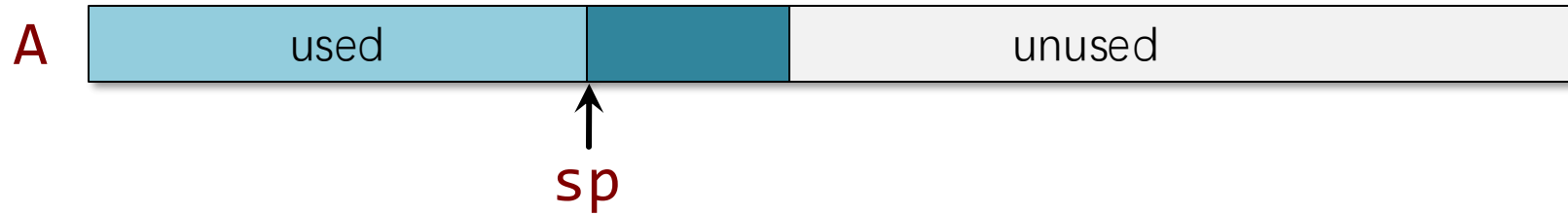
Array and Pointer: Allocating



Allocate x bytes

```
sp += x;  
return sp - x;
```

Array and Pointer: Allocating

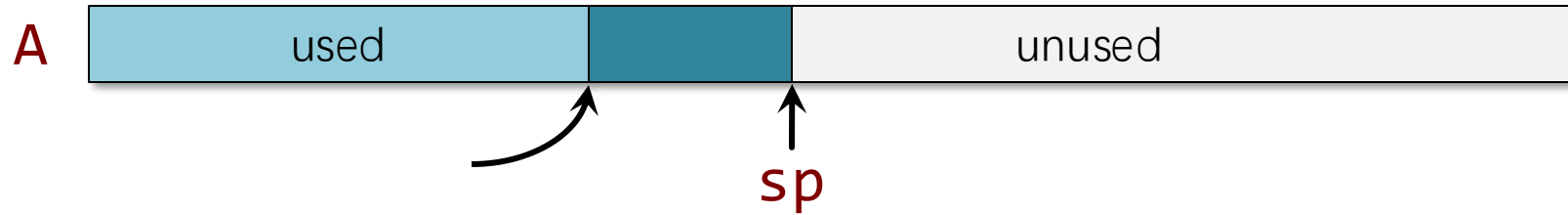


Allocate x bytes

```
sp += x;  
return sp - x;
```

Should check for
stack overflow.

Array and Pointer: Allocating

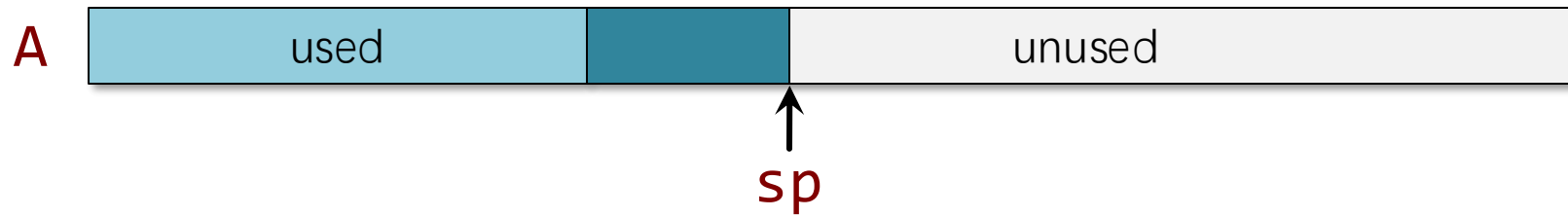


Allocate x bytes

```
sp += x;  
return sp - x;
```

No math if stack grows downward, but it doesn't really matter, because integer arithmetic is fast, and the processor core has many ALU's.

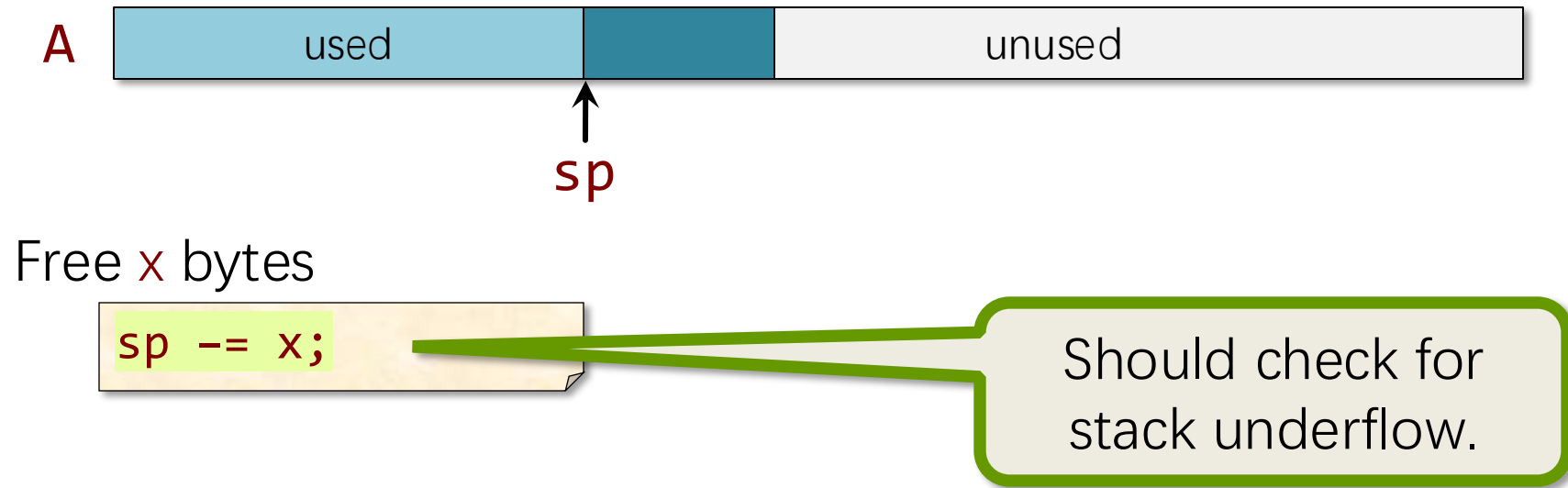
Array and Pointer: Deallocating



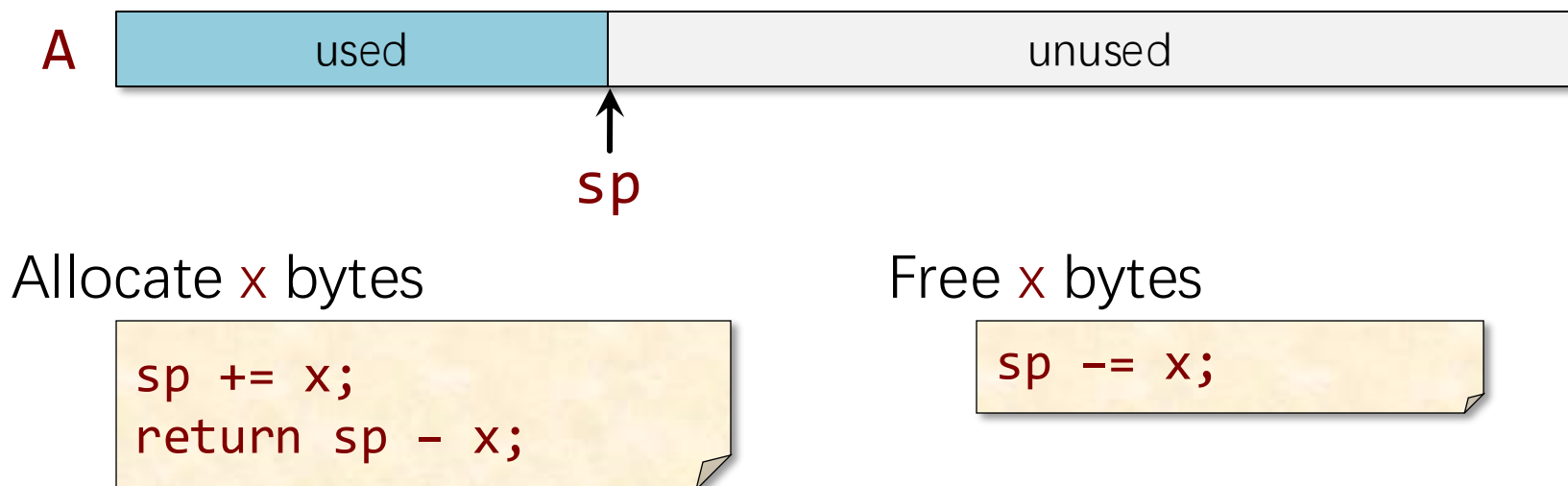
Free x bytes

```
sp -= x;
```

Array and Pointer: Deallocating

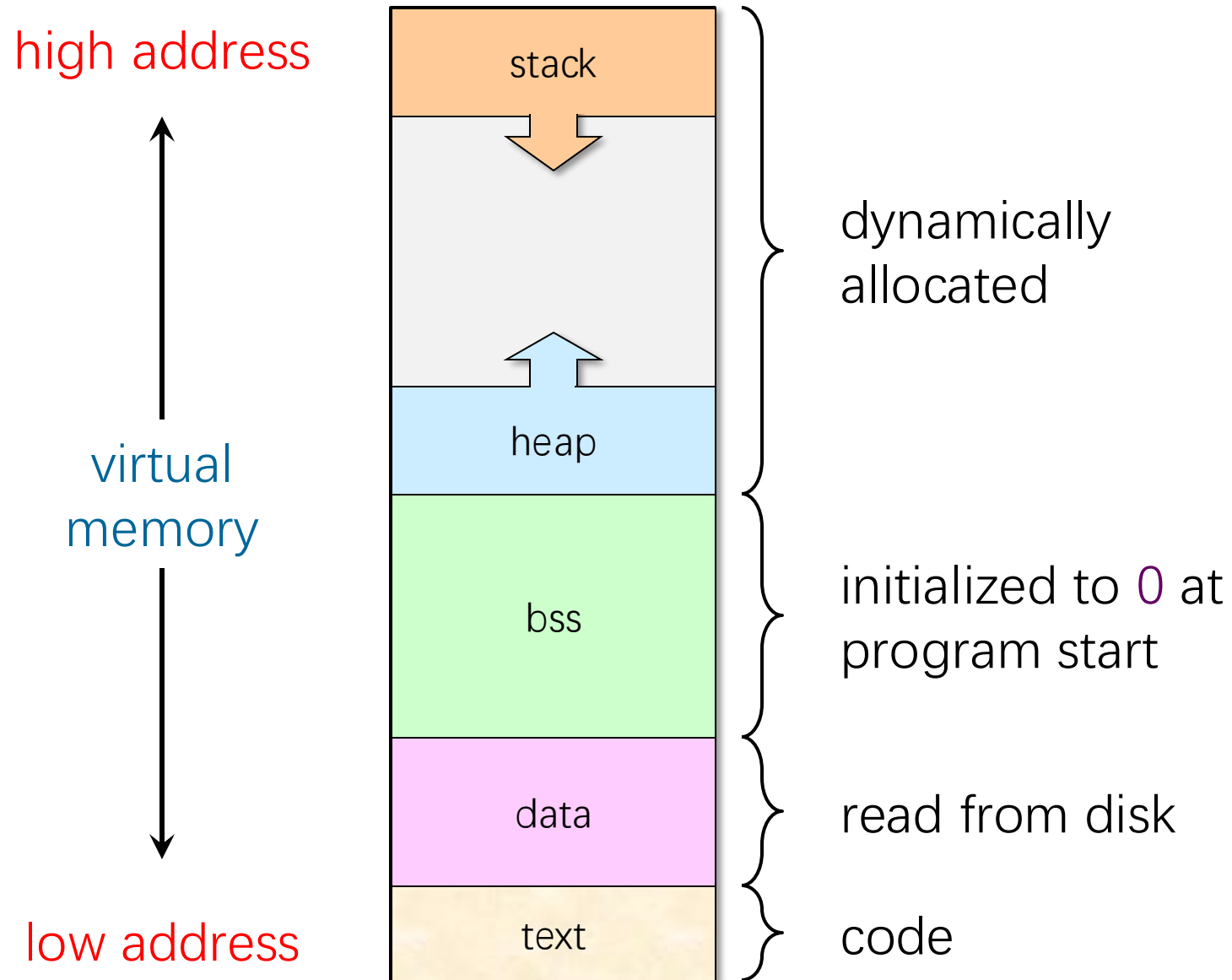


Summary of Stacks



- Allocating and freeing take $\Theta(1)$ time.
- Must free consistent with stack discipline.
- Limited applicability, but great when it works!
- Can dynamically allocate on the call stack using **alloca()**.
 - but this function is deprecated, and the compiler is more efficient with fixed-size frames

Program Segments



C Call Stack

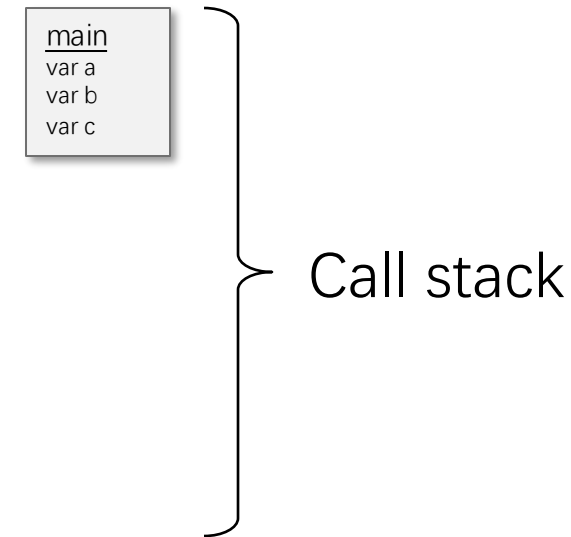
C call stack

- Stores the local variables for function instantiations.

C Call Stack

C call stack

- Stores the local variables for function instantiations.
- A frame is pushed onto the stack when the function is called.



```
int sub3(int a, int b) {  
    return (a - b) * (a + b);  
}
```

```
int sub2(int a, int d) {  
    return (a * d) + 2 * (a - d);  
}
```

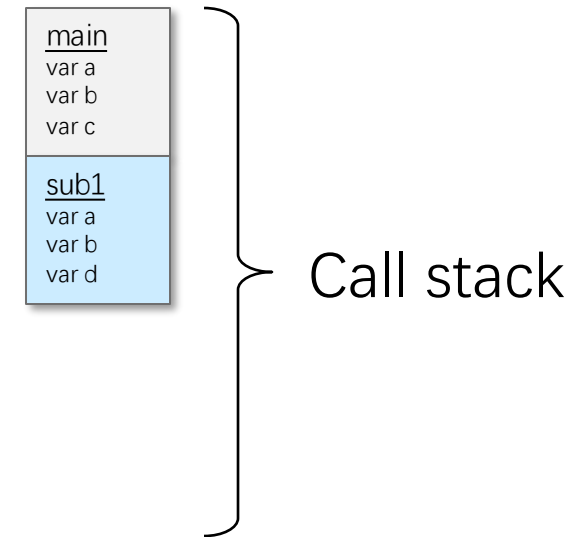
```
int sub1(int a, int b) {  
    int d = sub3(a, b);  
    return sub2(a, d);  
}
```

```
void main() {  
    int a, b;  
    a = 10;  
    b = 11;  
    return 190 + sub1(a,b);  
}
```

C Call Stack

C call stack

- Stores the local variables for function instantiations.
- A frame is pushed onto the stack when the function is called.



```
int sub3(int a, int b) {  
    return (a - b) * (a + b);  
}
```

```
int sub2(int a, int d) {  
    return (a * d) + 2 * (a - d);  
}
```

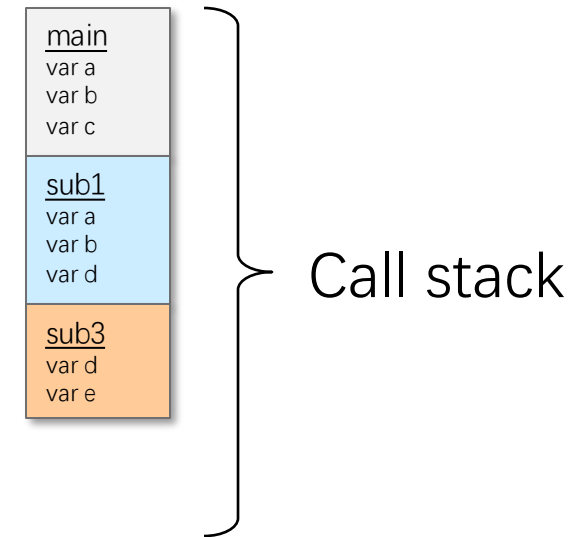
```
→ int sub1(int a, int b) {  
    int d = sub3(a, b);  
    return sub2(a, d);  
}
```

```
void main() {  
    int a, b;  
    a = 10;  
    b = 11;  
    return 190 + sub1(a,b);  
}
```

C Call Stack

C call stack

- Stores the local variables for function instantiations.
- A frame is pushed onto the stack when the function is called.



```
int sub3(int a, int b) {  
    return (a - b) * (a + b);  
}
```

```
int sub2(int a, int d) {  
    return (a * d) + 2 * (a - d);  
}
```

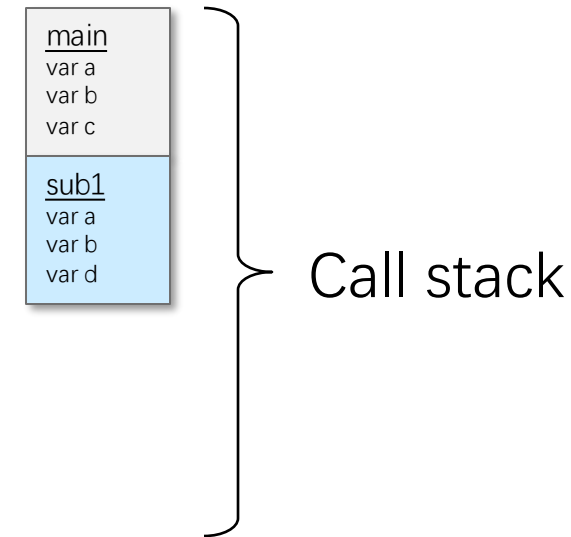
```
int sub1(int a, int b) {  
    int d = sub3(a, b);  
    return sub2(a, d);  
}
```

```
void main() {  
    int a, b;  
    a = 10;  
    b = 11;  
    return 190 + sub1(a, b);  
}
```

C Call Stack

C call stack

- Stores the local variables for function instantiations.
- A frame is pushed onto the stack when the function is called.
- The frame is popped when the function returns.



```
int sub3(int a, int b) {  
    return (a - b) * (a + b);  
}
```

```
int sub2(int a, int d) {  
    return (a * d) + 2 * (a - d);  
}
```

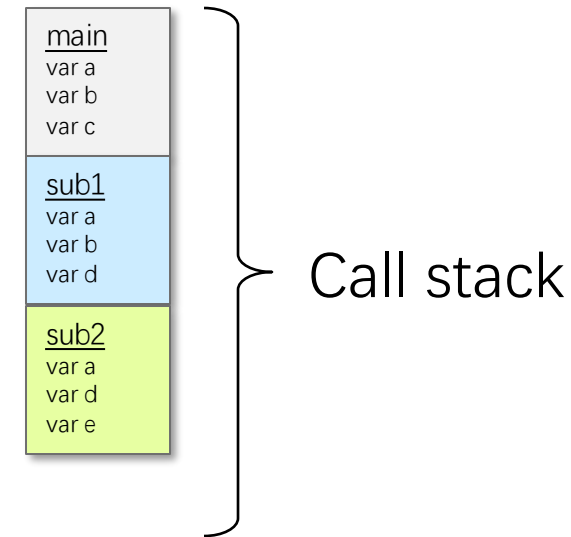
```
int sub1(int a, int b) {  
    int d = sub3(a, b);  
    return sub2(a, d);  
}
```

```
void main() {  
    int a, b;  
    a = 10;  
    b = 11;  
    return 190 + sub1(a,b);  
}
```

C Call Stack

C call stack

- Stores the local variables for function instantiations.
- A frame is pushed onto the stack when the function is called.
- The frame is popped when the function returns.



```
int sub3(int a, int b) {  
    return (a - b) * (a + b);  
}
```

```
int sub2(int a, int d) {  
    return (a * d) + 2 * (a - d);  
}
```

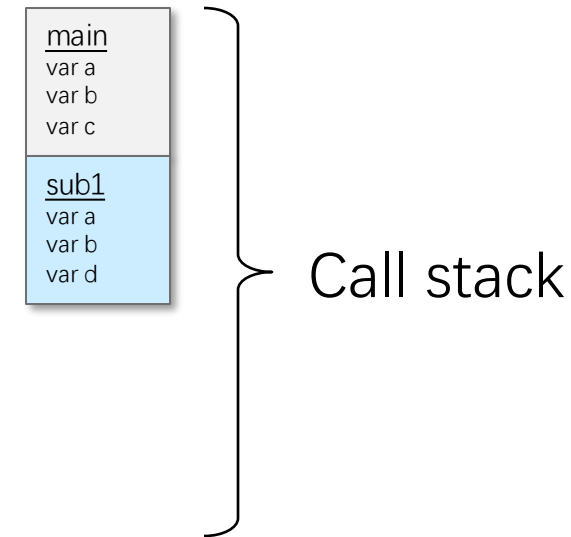
```
int sub1(int a, int b) {  
    int d = sub3(a, b);  
    return sub2(a, d);  
}
```

```
void main() {  
    int a, b;  
    a = 10;  
    b = 11;  
    return 190 + sub1(a, b);  
}
```

C Call Stack

C call stack

- Stores the local variables for function instantiations.
- A frame is pushed onto the stack when the function is called.
- The frame is popped when the function returns.



```
int sub3(int a, int b) {  
    return (a - b) * (a + b);  
}
```

```
int sub2(int a, int d) {  
    return (a * d) + 2 * (a - d);  
}
```

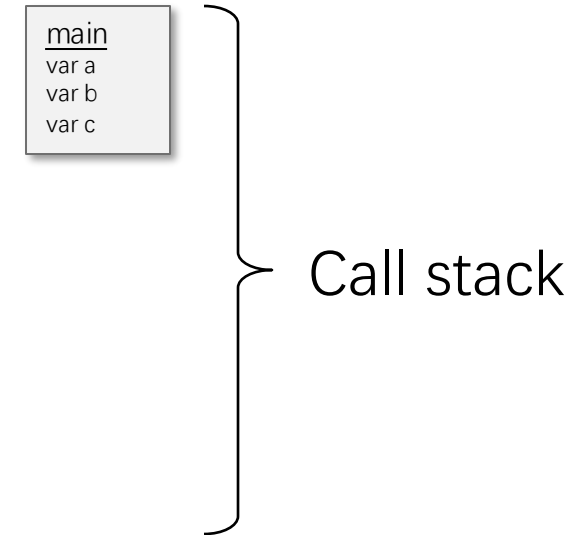
```
int sub1(int a, int b) {  
    int d = sub3(a, b);  
    return sub2(a, d);  
}
```

```
void main() {  
    int a, b;  
    a = 10;  
    b = 11;  
    return 190 + sub1(a, b);  
}
```


C Call Stack

C call stack

- Stores the local variables for function instantiations.
- A frame is pushed onto the stack when the function is called.
- The frame is popped when the function returns.



```
int sub3(int a, int b) {  
    return (a - b) * (a + b);  
}
```

```
int sub2(int a, int d) {  
    return (a * d) + 2 * (a - d);  
}
```

```
int sub1(int a, int b) {  
    int d = sub3(a, b);  
    return sub2(a, d);  
}
```

```
void main() {  
    int a, b;  
    a = 10;  
    b = 11;  
    return 190 + sub1(a,b);  
}
```

Summary: Stacks



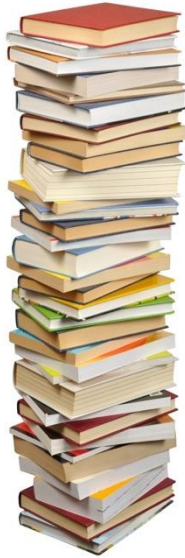
Stack

- Stacks are simple, fast, and easy to automate
 - ▣ compiler manages the stack for you
- Constrained to this specific structure of allocation
 - ▣ You can only free the last thing you allocated

What if you want to keep around the objects that you just allocated, and free things you allocated farther in the past?

Dynamic Storage Allocation

Kinds of storage management



Stack



Heap

C: `malloc()`, `free()`

C++: `new`, `delete`

HEAPS



Heap Storage in C

- Allocation

```
void* malloc(size_t s);
```

Effect: Allocate and return a pointer to a block of memory containing at least `s` bytes.

- Deallocation

```
void free(void *p);
```

Effect: `p` is a pointer to a block of memory returned by `malloc()` or `memalign()`.
Deallocate the block.

- Aligned allocation

```
void* memalign(size_t a, size_t s);
```

Effect: Allocate and return a pointer to a block of memory containing at least `s` bytes, aligned to a multiple of `a`, where `a` must be an exact power of 2:

```
assert(0==((size_t) memalign(a, s))%a) .
```

How `malloc` is Implemented?

- `glibc`'s `malloc()` relies on two system calls provided by OS
- `sbrk()` For small allocations (typically < 128 KB)
 - ▣ uses chunk-based allocation within pre-allocated memory obtained via `sbrk()`
- `mmap()` For large allocations (typically ≥ 128 KB)
 - ▣ directly maps memory using `mmap()`

mmap(): System Call Allocating Virtual Memory

`mmap()` is used to allocate virtual memory by **memory mapping**:

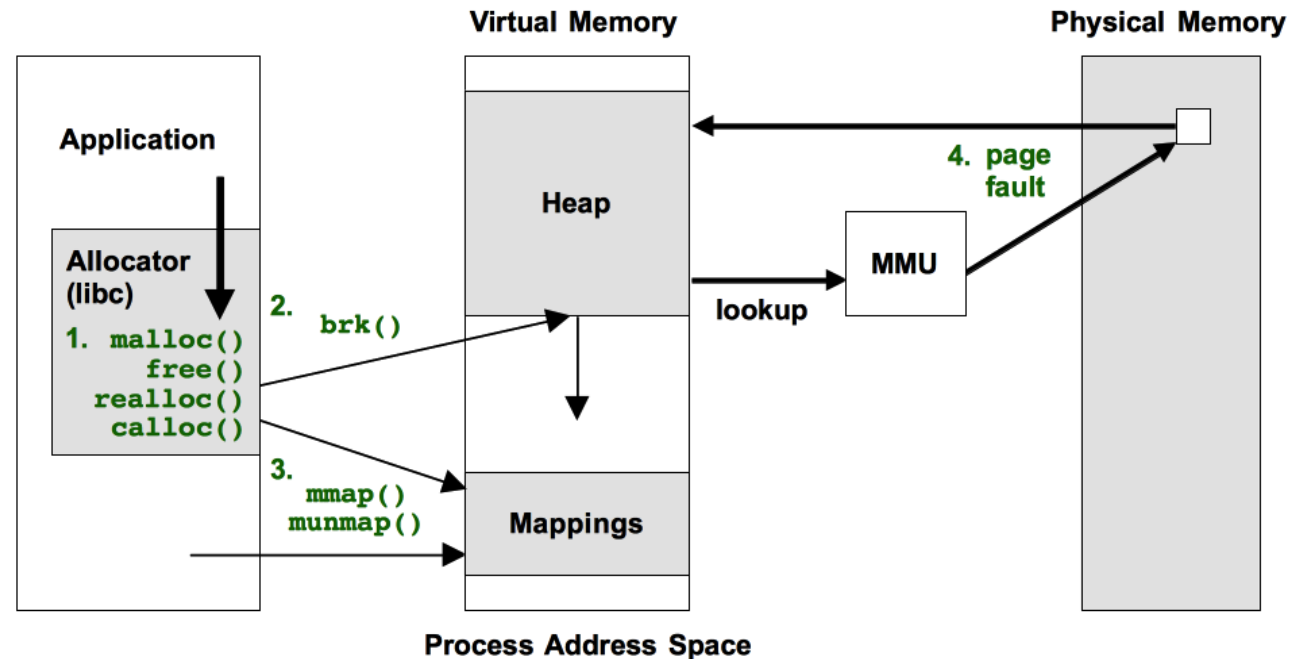
```
void *p = mmap(0,                // Don't care where
               size,             // #bytes
               PROT_READ | PROT_WRITE, // Read/write
               MAP_PRIVATE | MAP_ANON, // Private anonymous
               -1,               // no backing file
               0,               // offset (N/A)
               );
```

The Linux kernel

- **finds** a contiguous, unused region in the address space of the application large enough to hold size bytes,
- **modifies** the page table, and
- **creates** the necessary VM management structures within the OS to make the user's accesses to this area “legal” so that accesses won't segfault.

malloc() vs. mmap()

- **malloc()** and **free()** are part of **heap-management** code in the C library, e.g. **glibc**
- The heap-management code uses available system facilities, including **mmap()**, to obtain memory (virtual address space) from the kernel



malloc() vs. mmap()

- Why do we have both? Why don't we just call `mmap()` all the time?
- The heap-management code within `malloc()` attempts to satisfy user requests for heap storage by **reusing** freed memory whenever possible
- When necessary, the `malloc()` implementation invokes `mmap()` and other system calls to **expand** the size of the user's heap storage

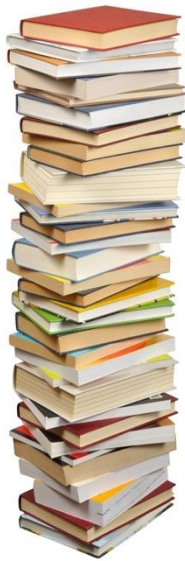
Heap Allocation*

- Heap space can be allocated and deallocated without constraint, but must be freed explicitly
 - ▣ C provides `malloc()` and `free()`
 - ▣ C++ provides `new` and `delete`
- Failure to `free` / `delete` creates a `memory leak`.
- Watch out for `dangling pointers` (pointers to freed memory) and `double freeing` (freeing memory that has already been freed)
- `Memory checkers` (e.g., `valgrind`) can assist in finding pernicious bugs. Use them!

*Do not confuse with a `heap data structure`.

Dynamic Storage Allocation

Kinds of storage management



Stack

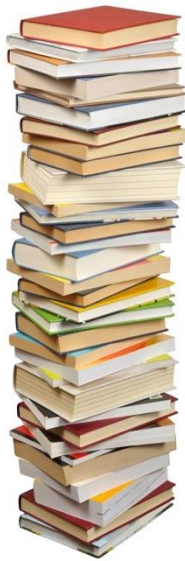


Explicitly Freed

Heap

Dynamic Storage Allocation

Kinds of storage management



Stack



Explicitly Freed



Garbage-Collected

Heap

Garbage Collection

- Garbage-collected storage need not be freed explicitly
 - improves programmer productivity and eliminates common bugs
- Available in most higher-level languages (e.g., Python, Java, Julia)
- GC looks for storage that the program can no longer access and **reclaims** it
- GC can pause the executing program, run in real time, or operate concurrently
- Garbage collection is usually slower than **malloc()** and **free()**
 - because the GC has to walk over your data to figure out, and
 - allocated storage is rarely in the L1-cache

Outline

- Virtual Memory System
- Stacks
- Heaps
 - ▣ Metrics to evaluate Allocators
 - ▣ Fixed-size Heap Allocation
 - ▣ Variable-size Heap Allocation
 - ▣ Parallel Heap Allocation

METRICS TO EVALUATE ALLOCATORS



Allocator Speed

Definition. Allocator **speed** is the number of allocations and deallocations per second that the allocator can sustain

Q. Is it more important to maximize allocator speed for large blocks or small blocks?

A. Small blocks!

Q. Why?

Hint. Typically, a user program writes all the bytes of an allocated block.

A. A large block takes so much time to write that the allocator time has little effect on the overall runtime. In contrast, if a program allocates many small blocks, the allocator time is a significant overhead.

Fragmentation

Definition.

user footprint: the maximum over time of the number of bytes, M , in use by the user program (allocated but not freed).

allocator footprint: is the maximum over time of the number of bytes, H , provided to the allocator by the operating system.

fragmentation: is $F = H/M$

space utilization: is M/H .

Remark. H grows monotonically with time for many allocators.

Remark. Modern 64-bit processors provide about 2^{48} bytes of virtual address space. A big server might have 2^{40} bytes of physical memory.

Fragmentation Glossary

- **Space overhead**: Space used by the allocator for bookkeeping.
- **Internal fragmentation**: Waste due to allocating larger blocks than the user requests.
- **External fragmentation**: Waste due to the inability to use storage because it is not contiguous.
- **Blowup**: For a parallel allocator, the additional space beyond what a serial allocator would require.

FIXED-SIZE HEAP ALLOCATION



Bitmap Allocator



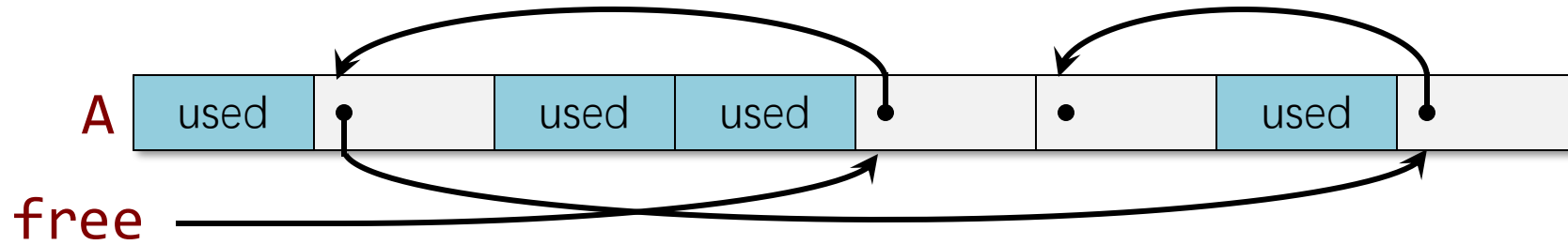
bitmap: 01001101

- Use a bitmap to keep track of which blocks of **A** are free & which are used
- Block sizes can be arbitrarily small
- Some tricks can improve things
 - ▣ Bit tricks: e.g., **bitmap & (-bitmap)** — but the approach is fundamentally not scalable (linear-time search)
 - ▣ Architecture-specific Intrinsics: e.g., **_mm_tzcnt** * which counts the number of trailing zeroes in a bitmap (≤ 64 bits)
 - ▣ Multilayer hierarchy: bitmap per page and bitmap for pages

* tzcnt intrinsic that counts trailing zeros – can be used to find lowest bit index with a 1.

https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html#text=_mm_tzcnt&ig_expand=4186,6903

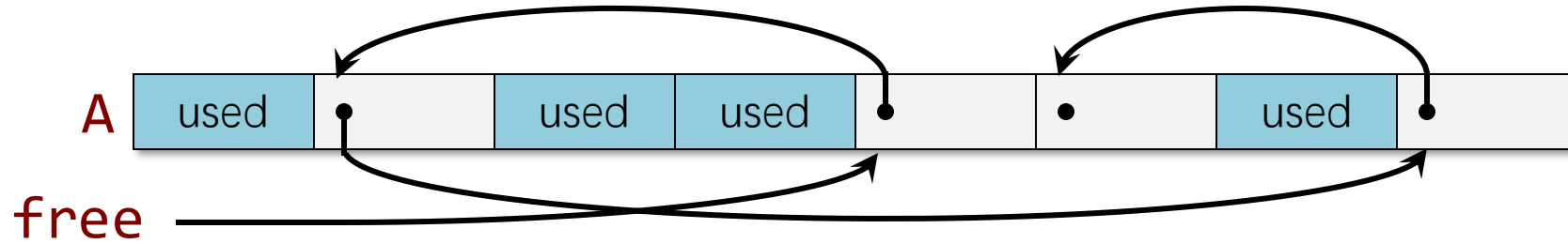
Free List



- Every piece of storage has the same size
- Each unused storage block contains a pointer to the next unused block
 - ▣ The block size must be at least as big as a pointer

```
struct freelist_item {  
    void *next;  
}
```

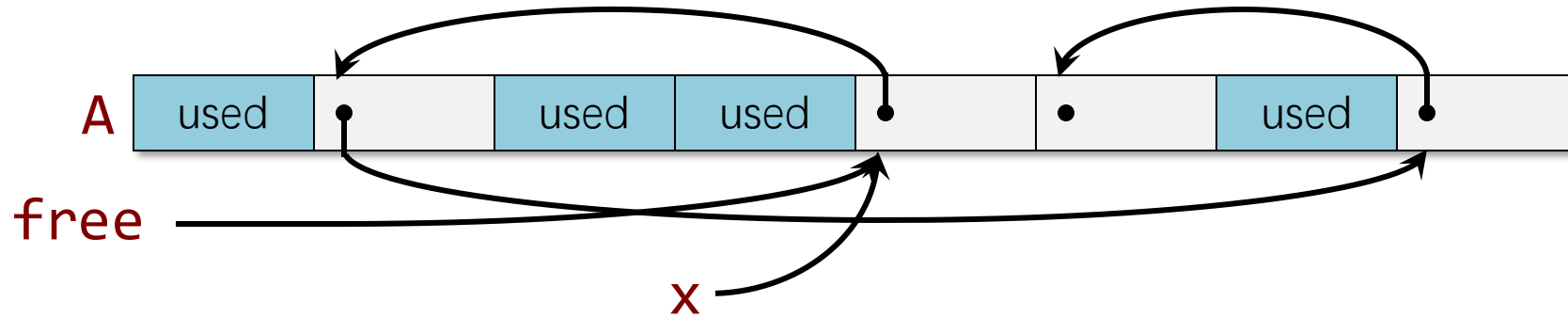
Free-List: Allocating



Allocate 1 object

```
x = free;  
free = free->next;  
return x;
```

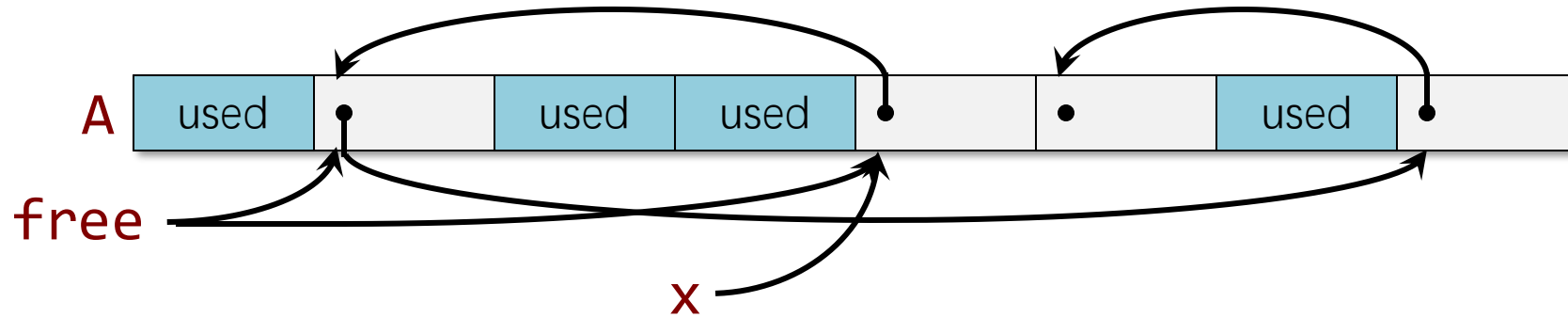
Free-List: Allocating



Allocate 1 object

```
x = free;  
free = free->next;  
return x;
```

Free-List: Allocating

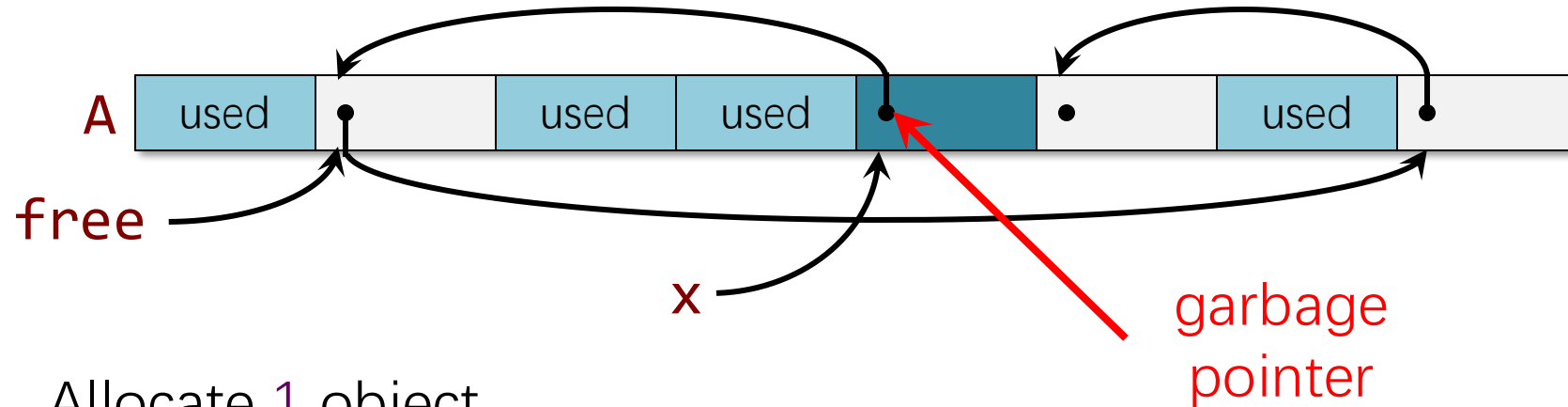


Allocate 1 object

```
x = free;  
free = free->next;  
return x;
```

Should check **free** **!=** NULL.

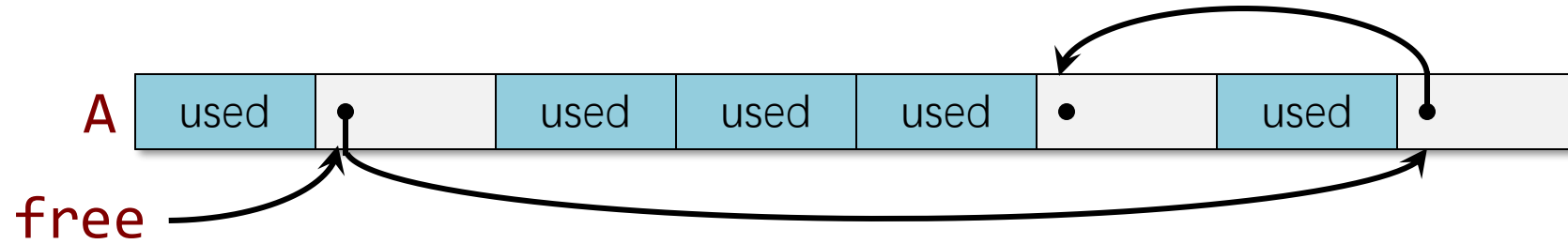
Free-List: Allocating



Allocate 1 object

```
x = free;  
free = free->next;  
return x;
```

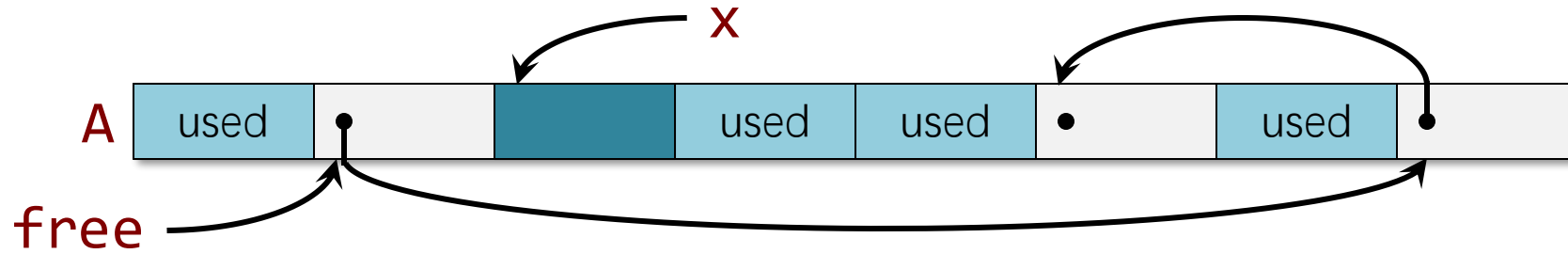
Free-List: Allocating



Allocate 1 object

```
x = free;  
free = free->next;  
return x;
```

Free-List: Deallocating



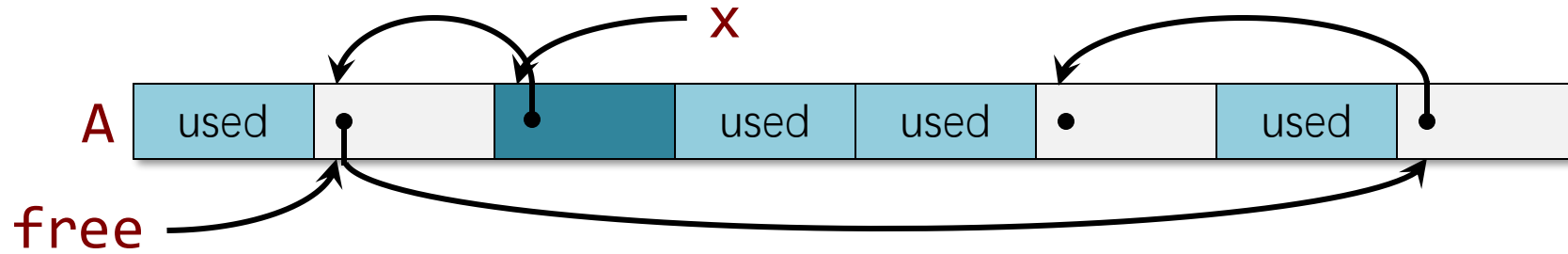
Allocate 1 object

```
x = free;  
free = free->next;  
return x;
```

free object x

```
x->next = free;  
free = x;
```

Free-List: Deallocating



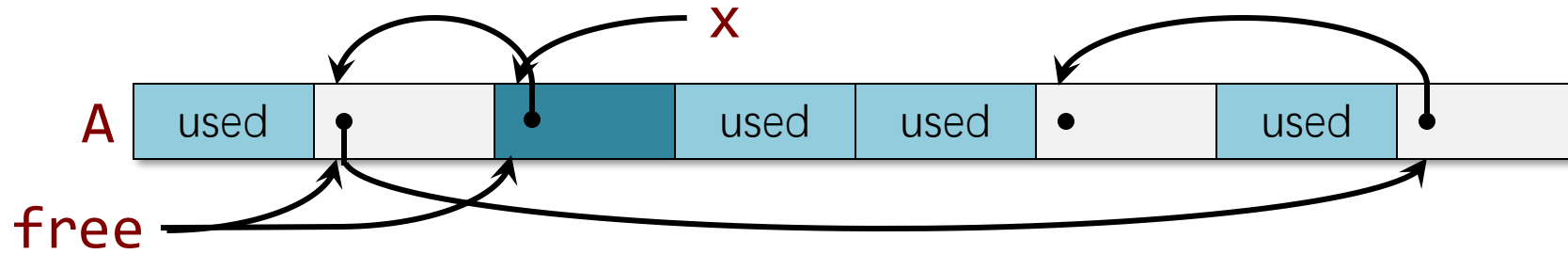
Allocate 1 object

```
x = free;  
free = free->next;  
return x;
```

free object x

```
x->next = free;  
free = x;
```

Free-List: Deallocating



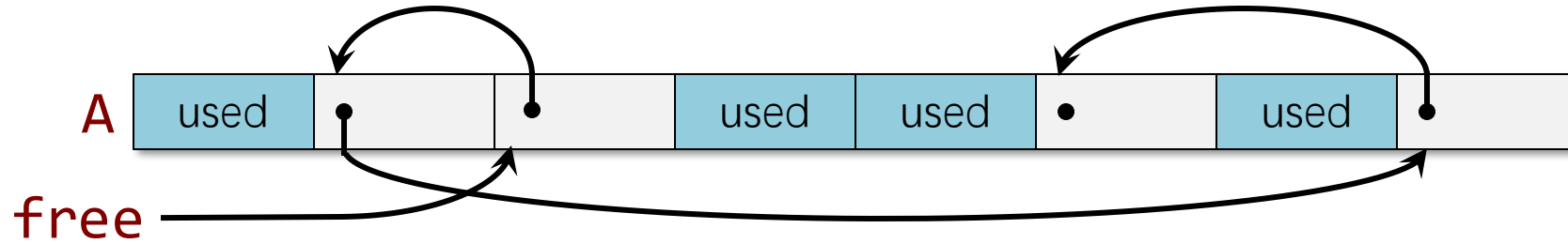
Allocate 1 object

```
x = free;  
free = free->next;  
return x;
```

free object x

```
x->next = free;  
free = x;
```

Free-List: Deallocating



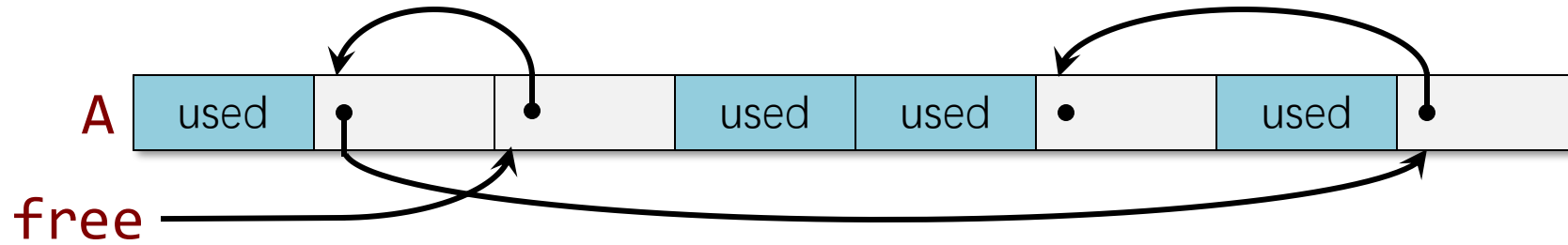
Allocate 1 object

```
x = free;  
free = free->next;  
return x;
```

free object **x**

```
x->next = free;  
free = x;
```

Summary of Free Lists



- Allocating and freeing take $\Theta(1)$ time.
- Good temporal locality.
- Poor spatial locality due to **external fragmentation**
 - ▣ can increase the size of the page table and cause **disk thrashing**.
- The **translation lookaside buffer** (TLB) can also be a problem

blocks distributed
across virtual
memory

Mitigating External Fragmentation

- Keep a free list (or bitmap) **per disk page**.
- Allocate from the free list for the fullest unfull page.
- To free a block, add it to the free list for the page on which the block resides
- If a page becomes **empty** (only free-list items), the virtual-memory system can **page it out** without substantial impact on program performance.
- 90-10 beats 50-50:



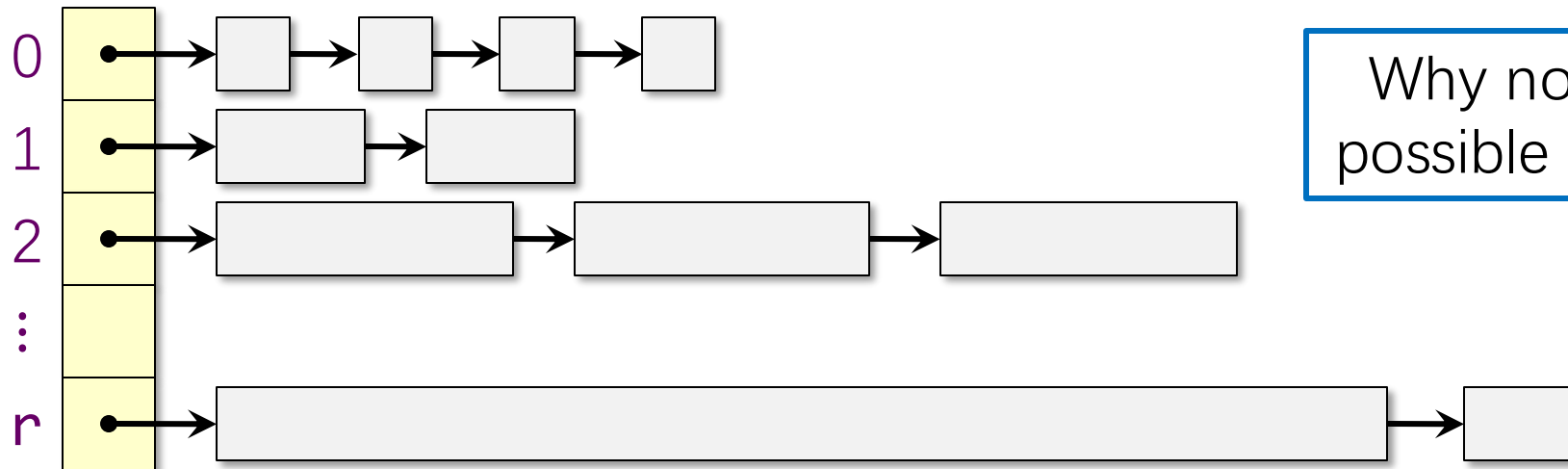
Probability that 2 random accesses hit the same page
 $= .9 \times .9 + .1 \times .1 = .82$ versus $.5 \times .5 + .5 \times .5 = .5$

VARIABLE-SIZE HEAP ALLOCATION



Binned Free Lists

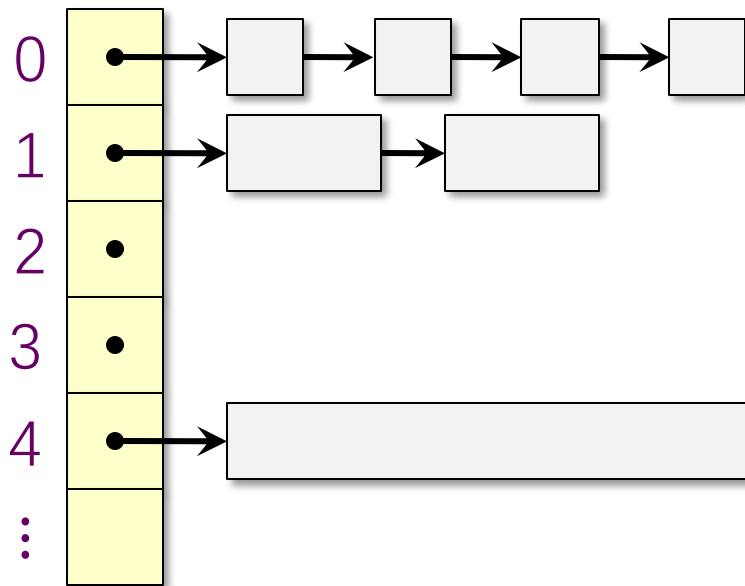
- Bin k holds memory blocks of size 2^k
- Leverage the efficiency of free lists
- Accept a bounded amount of internal fragmentation



Why not bins for every possible size? No IF then.

Binned Free Lists: Allocating

- Allocate x bytes
- If bin $k = \lceil \lg x \rceil$ is nonempty, return a block.
 - Otherwise, find a block in the next larger nonempty bin $k' > k$, split it up into blocks of sizes $2^{k'-1}, 2^{k'-2}, \dots, 2^k, 2^k$, and distribute the pieces.



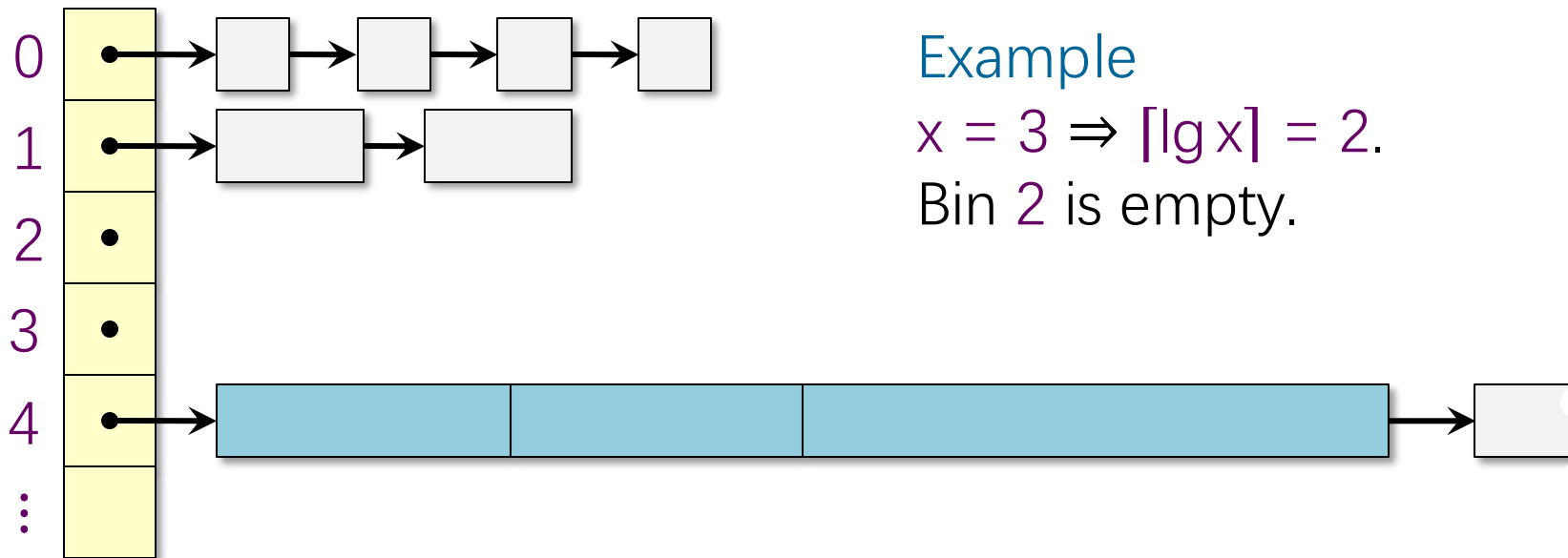
Example

$x = 3 \Rightarrow \lceil \lg x \rceil = 2.$

Bin 2 is empty.

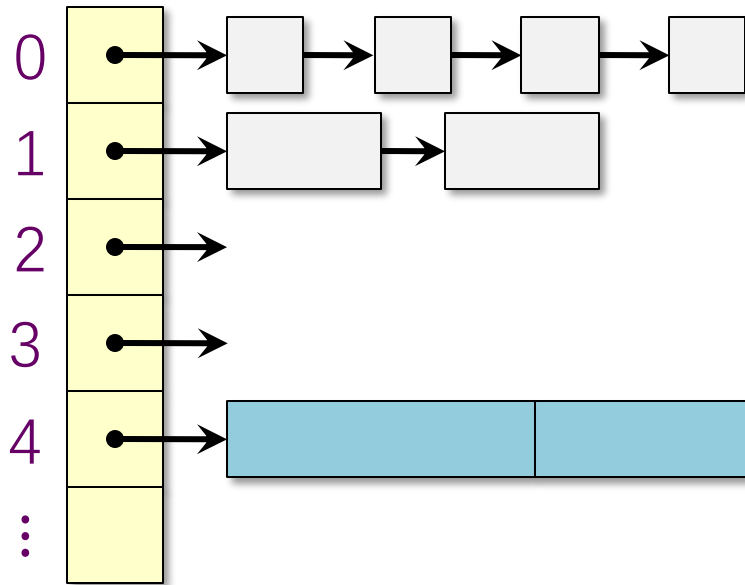
Binned Free Lists: Allocating

- Allocate x bytes
- If bin $k = \lceil \lg x \rceil$ is nonempty, return a block.
 - Otherwise, find a block in the next larger nonempty bin $k' > k$, split it up into blocks of sizes $2^{k'-1}, 2^{k'-2}, \dots, 2^k, 2^k$, and distribute the pieces.



Binned Free Lists: Allocating

- Allocate x bytes
- If bin $k = \lceil \lg x \rceil$ is nonempty, return a block.
 - Otherwise, find a block in the next larger nonempty bin $k' > k$, split it up into blocks of sizes $2^{k'-1}$, $2^{k'-2}$, $\dots$, 2^k , 2^k , and distribute the pieces.



Example

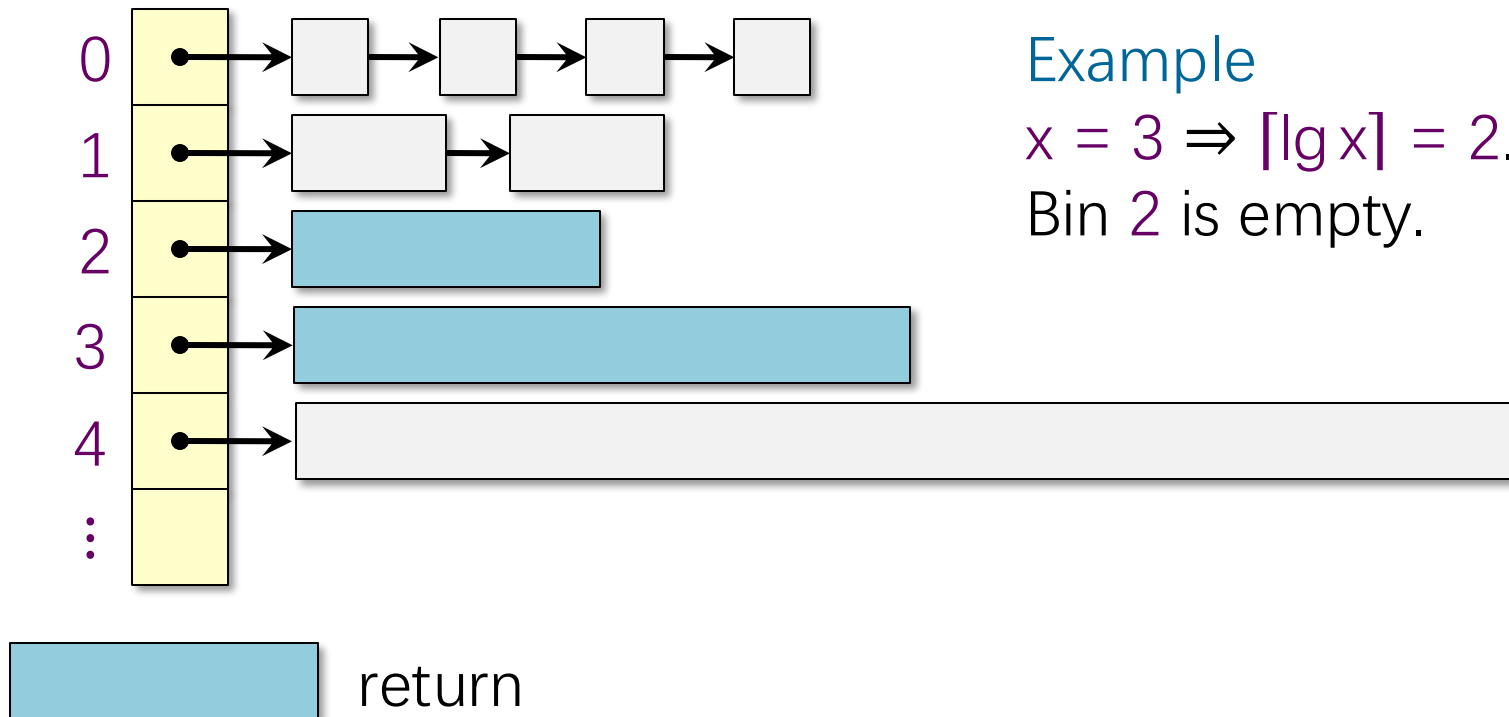
$x = 3 \Rightarrow \lceil \lg x \rceil = 2$.

Bin 2 is empty.

return

Binned Free Lists: Allocating

- Allocate x bytes
- If bin $k = \lceil \lg x \rceil$ is nonempty, return a block.
 - Otherwise, find a block in the next larger nonempty bin $k' > k$, split it up into blocks of sizes $2^{k'-1}$, $2^{k'-2}$, $\dots$, 2^k , 2^k , and distribute the pieces.
 - If no larger blocks exist, ask OS to allocate more memory



How Virtual is Virtual Memory?

Q. Since a 64-bit address space takes over 8 years to write at a rate of 64 gigabytes per second (GDDR6 technology), we effectively never run out of virtual memory. So, why not just allocate increasing VM addresses and never free?

A. External fragmentation would be horrendous! The performance of the page table would degrade tremendously leading to disk thrashing, since all nonzero memory must be backed up on disk in page-sized blocks.

Goal of storage allocators

Use as little virtual memory as possible,
and try to keep the used portions relatively **compact**.

Analysis of Binned Free Lists

• **Theorem.** Suppose that the maximum amount of heap memory in use at any time by a program is M . If the heap is managed by a BFL allocator, the amount of virtual memory consumed by heap storage is $O(M \lg M)$.

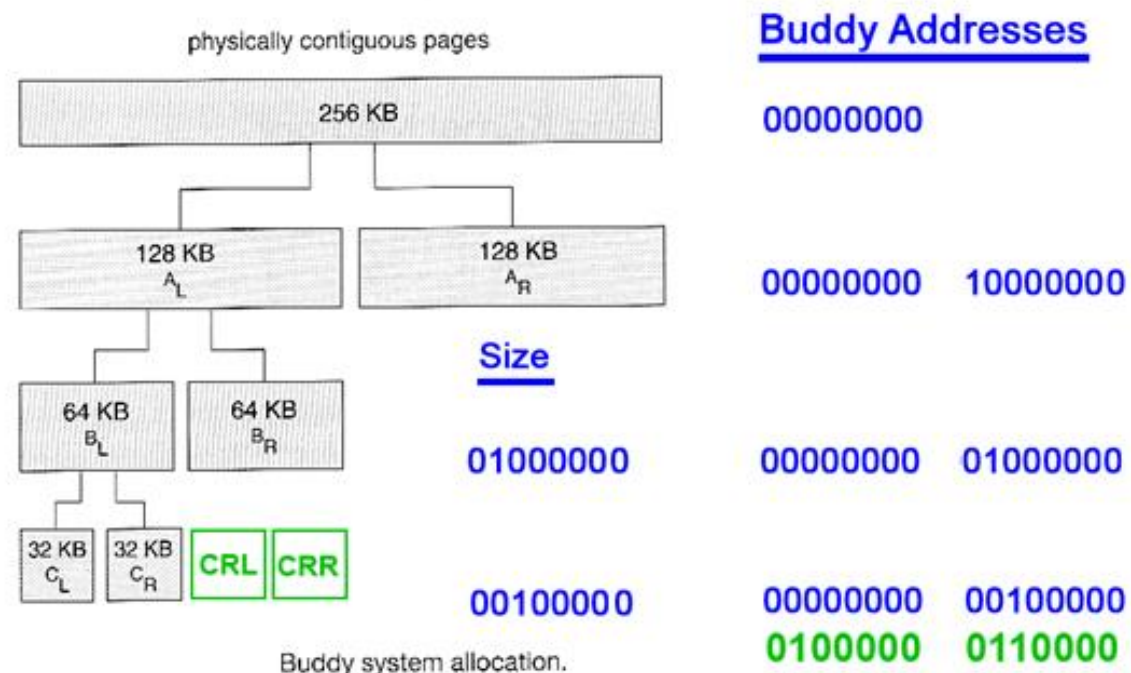
Proof. An allocation request for a block of size x consumes $2^{\lceil \lg x \rceil} \leq 2x$ storage. Thus, the amount of virtual memory devoted to blocks of size 2^k is at most $2M$. Since there are at most $\lg M$ free lists, the theorem holds. ■

⇒ In fact, BFL is 6-competitive with the optimal omniscient allocator.

Coalescing

Binned free lists can sometimes be heuristically improved by **splicing together** adjacent small blocks into a larger block.

- Clever schemes exist for finding adjacent blocks efficiently — e.g., the **“buddy” system** — but the overhead is still greater than simple BFL



Coalescing

Binned free lists can sometimes be heuristically improved by **splicing together** adjacent small blocks into a larger block.

- Clever schemes exist for finding adjacent blocks efficiently — e.g., the **“buddy” system** — but the overhead is still greater than simple BFL
- No good theoretical bounds exist that **prove** the effectiveness of coalescing
- Coalescing seems to reduce fragmentation **in practice**, because heap storage often obeys a stack discipline or tends to be deallocated in batches

PARALLEL HEAP ALLOCATION



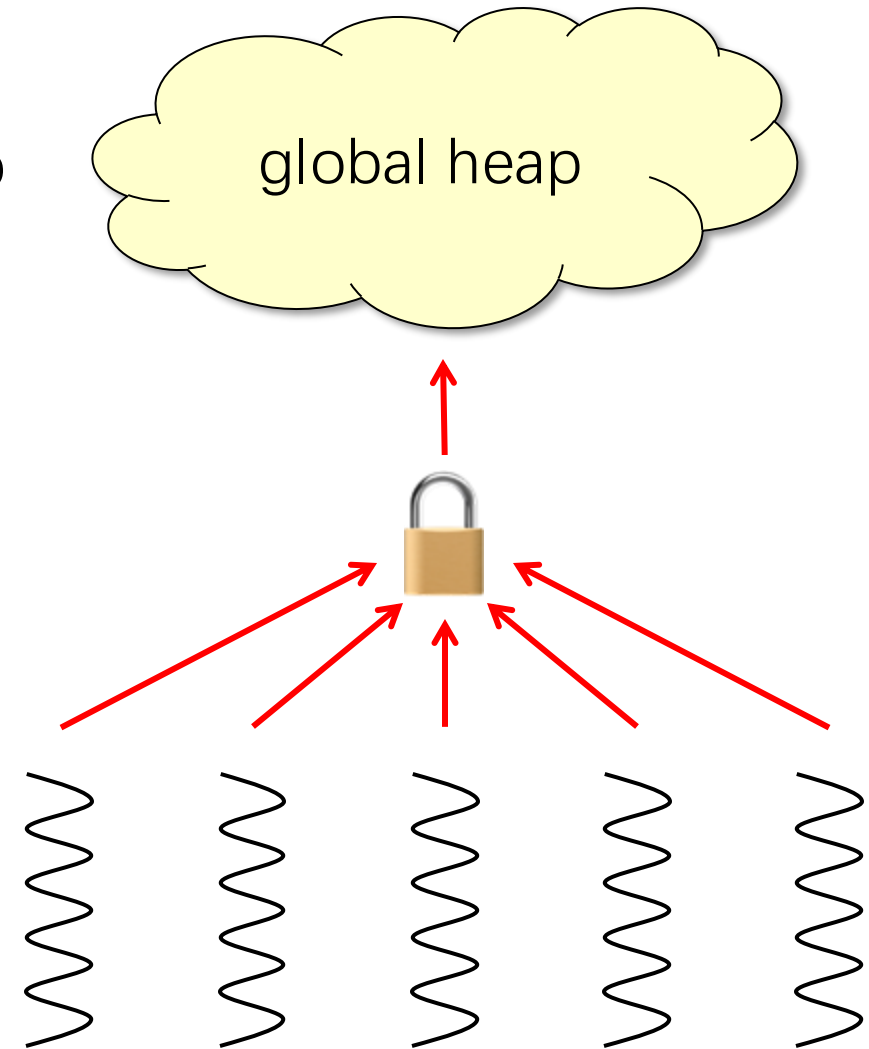
Strategy 1: Global Heap

- Default C allocator
- All threads (processors) share a single heap
- Accesses are mediated by a mutex (or lock-free synchronization) to preserve atomicity

😊 Blowup = 1.

☹️ Slow — acquiring a lock is like an L3-cache access.

☹️ Contention can inhibit scalability.



Scalability

- Ideally, as the number of threads (processors) grows, the time to perform an allocation or deallocation should not increase.
- The most common reason for loss of scalability is **lock contention**.

Q. Is lock contention more of a problem for large blocks or small blocks?

A. Small blocks!

Q. Why?

Hint. Typically, a user program writes all the bytes of an allocated block.

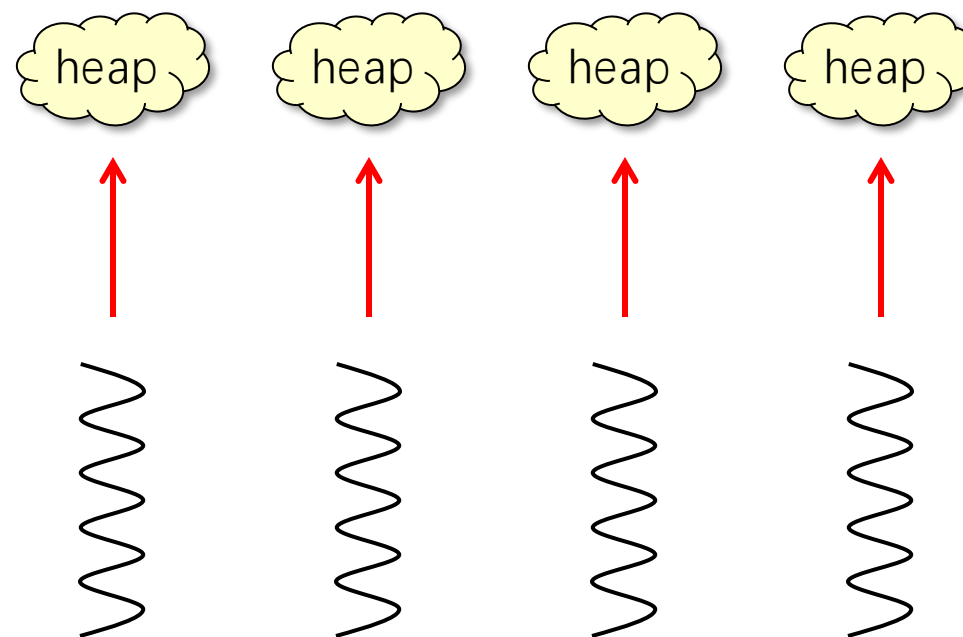
A. It's unlikely for a thread allocating large blocks to issue allocation requests at a high rate. In contrast, if a program allocates many small blocks in parallel, contention can be a significant issue.

Strategy 2: Local Heaps

- Each thread allocates out of its own heap
- No locking is necessary

- 😊 Fast — no synchronization.
- 😞 Suffers from **memory drift**:
blocks allocated by one thread
are freed on another

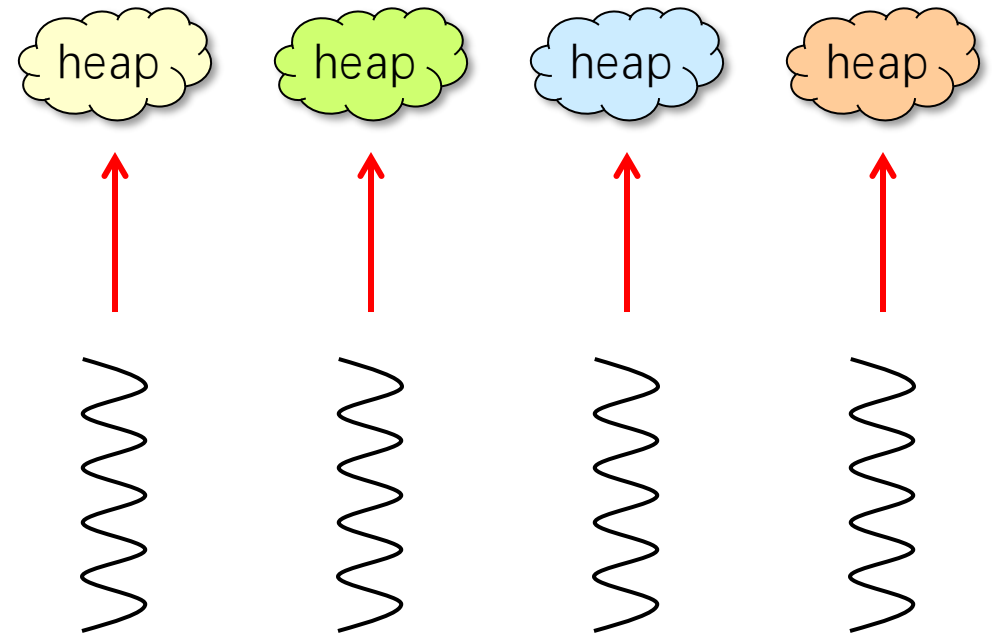
⇒ **unbounded blowup.**



Strategy 3: Local Ownership

- Each object is labeled with its owner
- Freed objects are returned to the owner's heap

- 😊 Fast allocation and freeing of local objects.
- 😞 Freeing remote objects requires synchronization.
- 😐 Blowup $\leq P$.
- 😊 Resilience to **false sharing**.

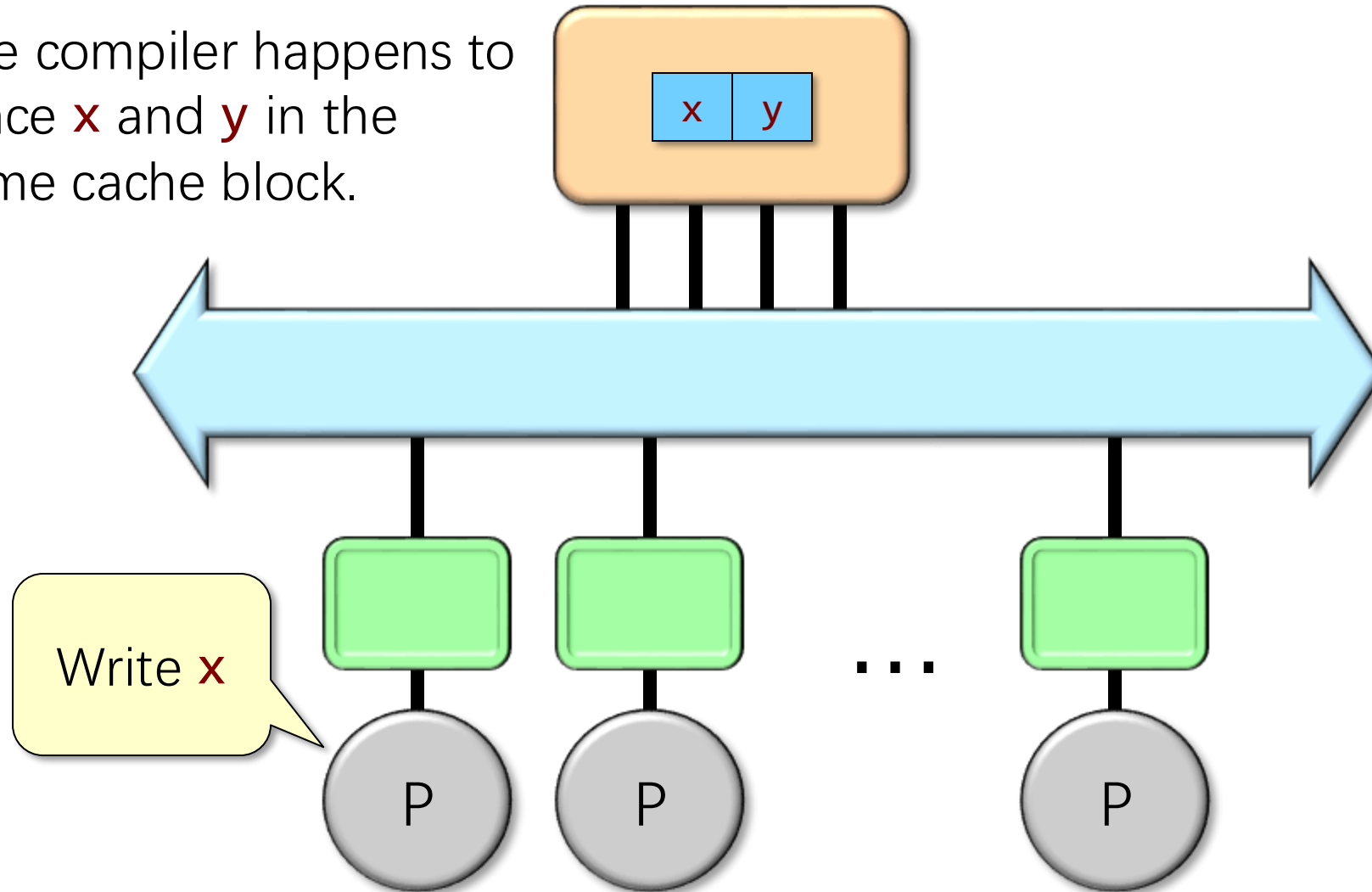


FALSE SHARING



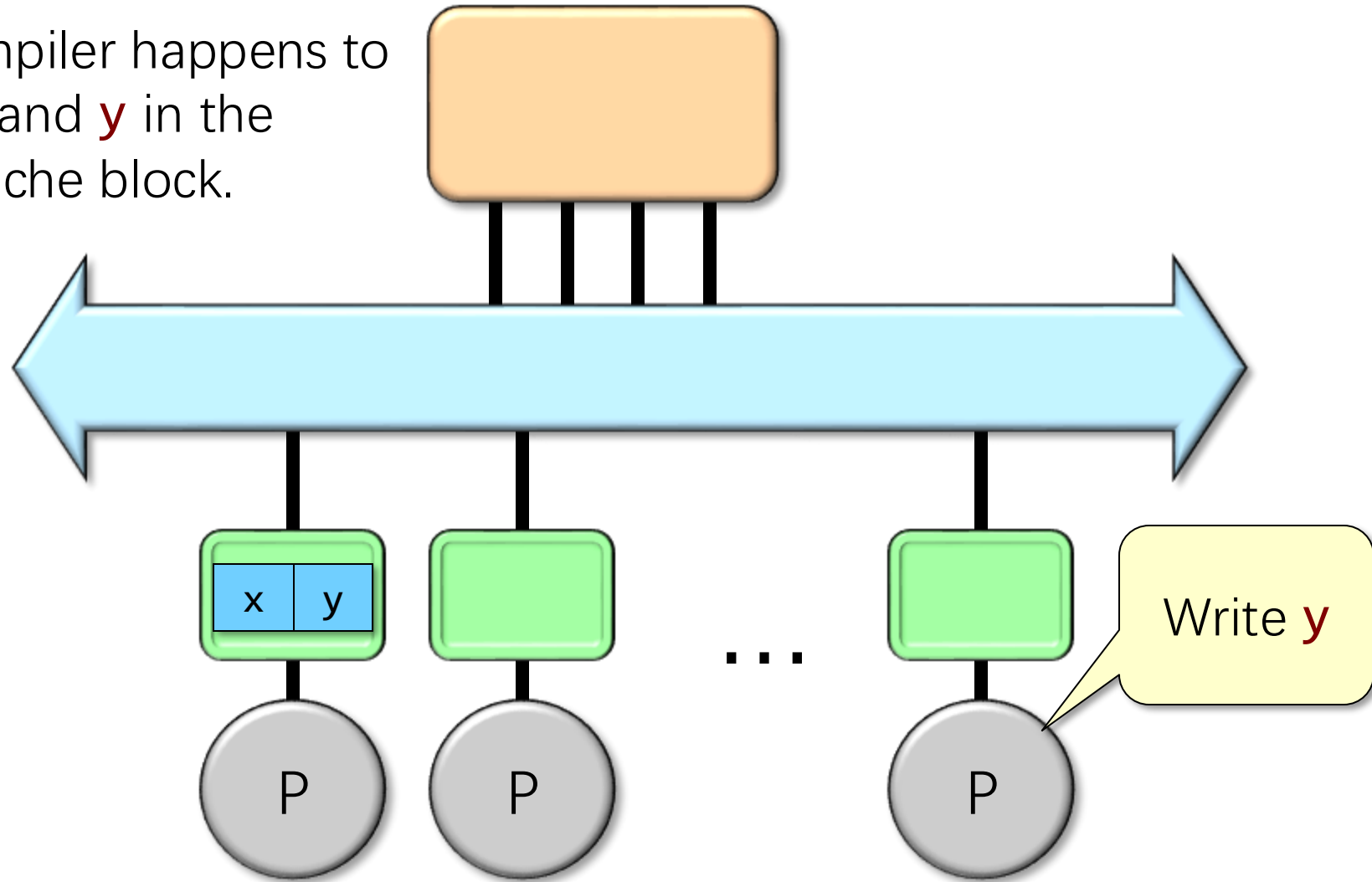
False Sharing Example

The compiler happens to place **x** and **y** in the same cache block.



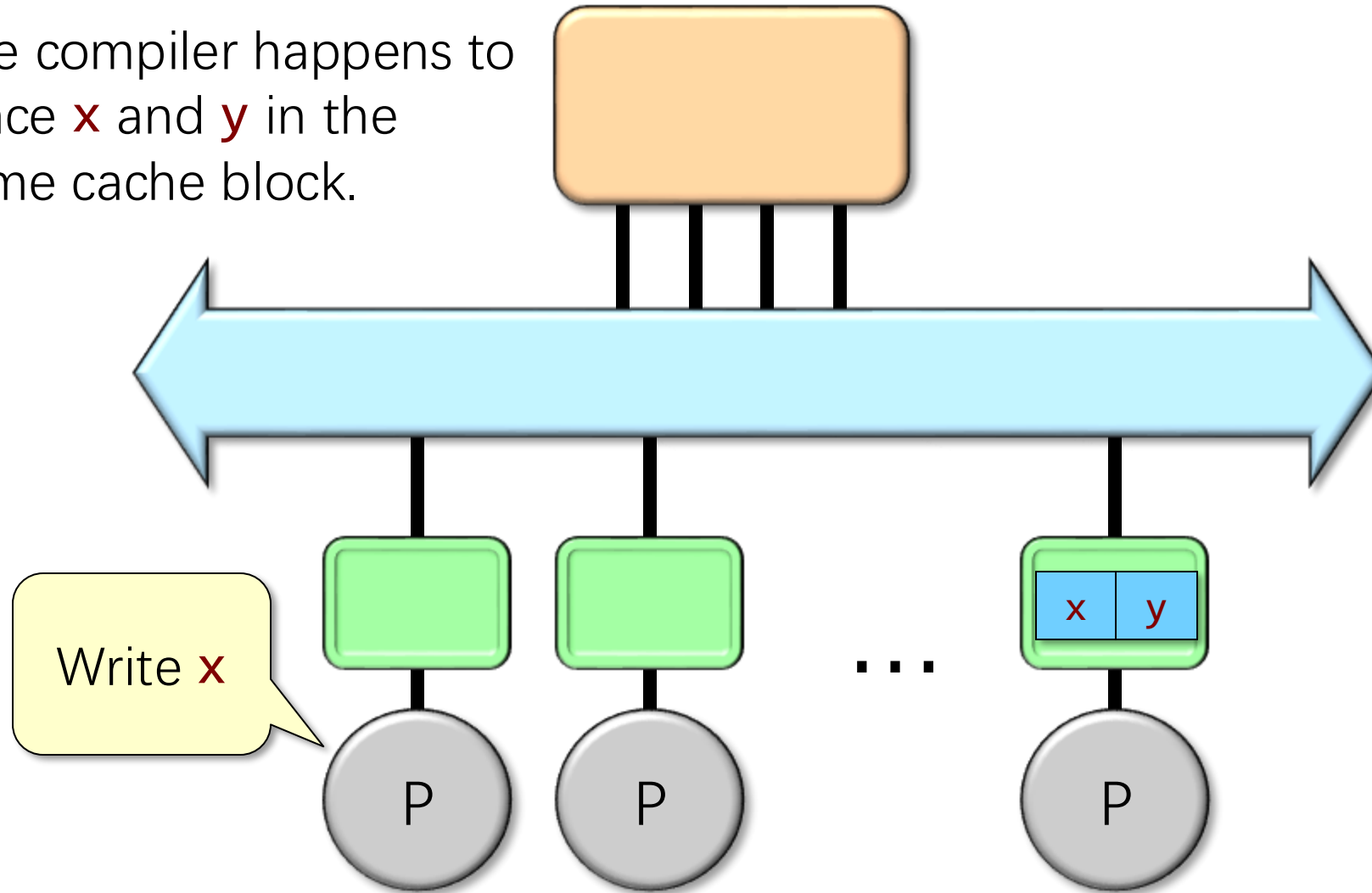
False Sharing Example

The compiler happens to place **x** and **y** in the same cache block.



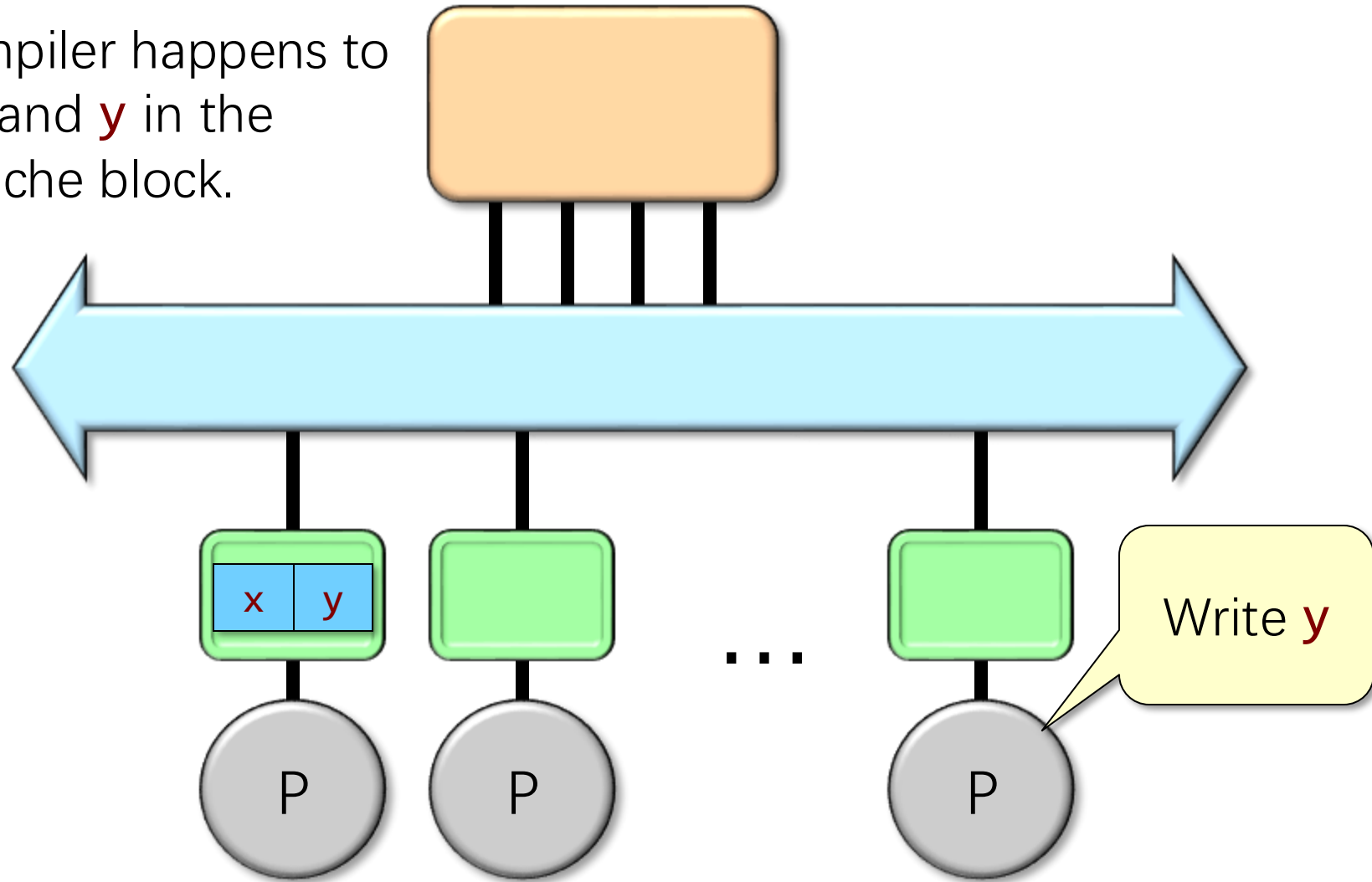
False Sharing Example

The compiler happens to place **x** and **y** in the same cache block.



False Sharing Example

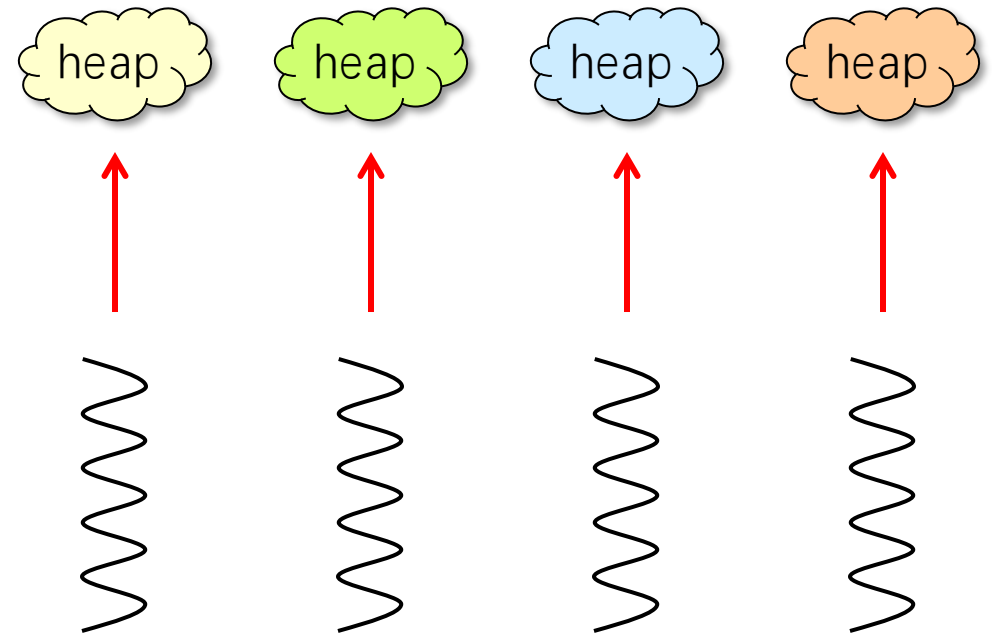
The compiler happens to place **x** and **y** in the same cache block.



Strategy 3: Local Ownership

- Each object is labeled with its owner
- Freed objects are returned to the owner's heap

- 😊 Fast allocation and freeing of local objects.
- 😞 Freeing remote objects requires synchronization.
- 😐 Blowup $\leq P$.
- 😊 Resilience to **false sharing**.



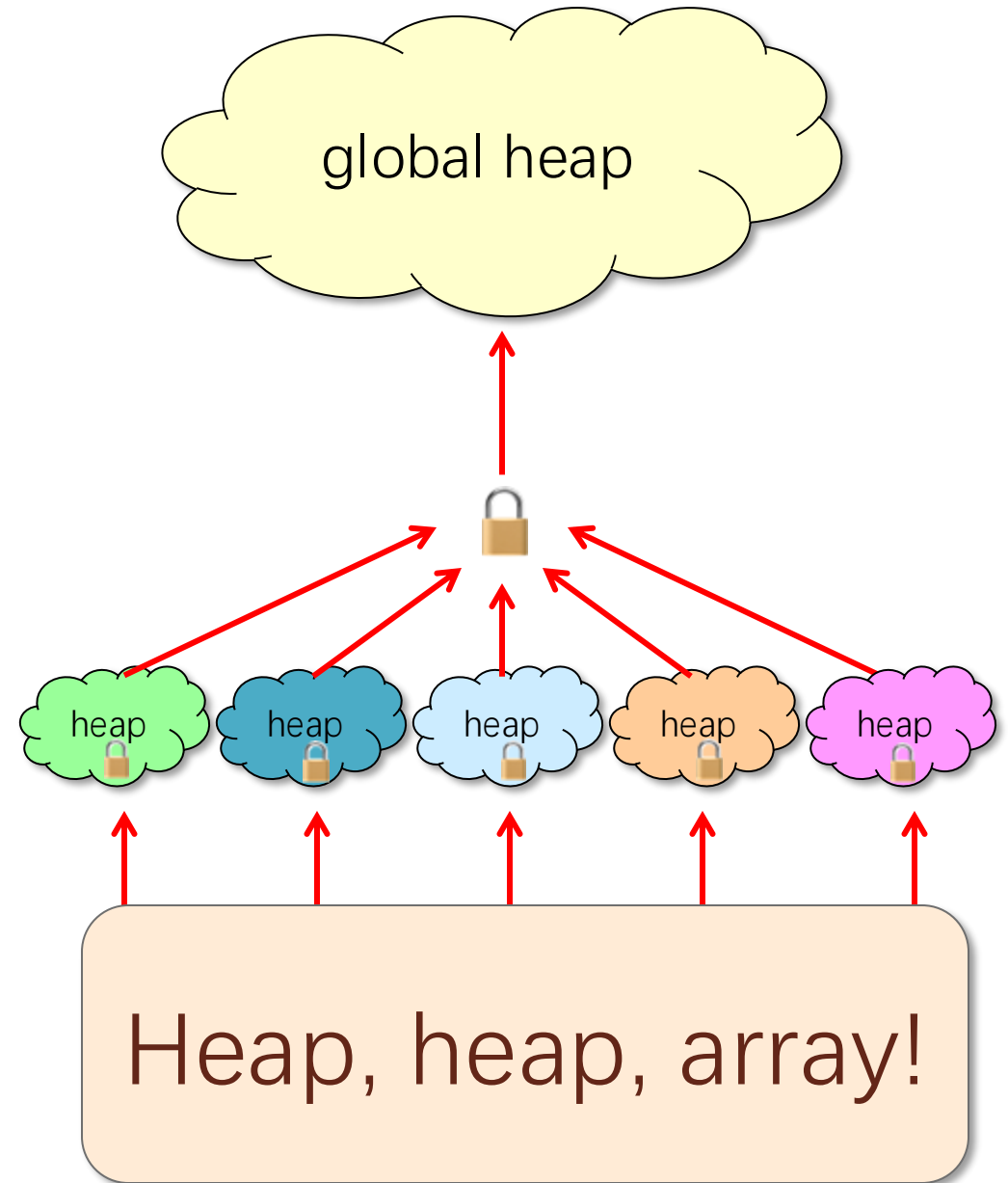
BACK TO PARALLEL HEAP ALLOCATION



The Hoard Allocator*

- P local heaps.
 - 1 global heap.
 - Memory is organized into large **superblocks** of size S .
 - Only superblocks are moved between the local heaps and the global heap.
- ☺ Fast.
- ☺ Scalable.
- ☺ Bounded blowup.
- ☺ Resilience to false sharing.

* See reading



Hoard Allocation

Assume without loss of generality that all blocks are the same size (fixed-size allocation).

x = malloc() on thread **i**:

```
if (there exists a free object in heap i) {  
    x = an object from the fullest nonfull superblock in i's heap;  
} else {  
    if (the global heap is empty) {  
        B = a new superblock from the OS;  
    } else {  
        B = a superblock in the global heap;  
    }  
    set the owner of B to i;  
    x = a free object in B;  
}  
return x;
```


Hoard Deallocation

Let m_i be the in-use storage in heap i , and let h_i be the storage owned by heap i .

Hoard maintains the following invariant for all heaps i :

$$m_i \geq \min(h_i - 2S, h_i/2),$$

where S is the superblock size.

free(x), where x is owned by thread i :

```
put  $x$  back in heap  $i$ ;  
if ( $m_i < \min(h_i - 2S, h_i/2)$ ) {  
    move a superblock that is at least half empty from  
    heap  $i$  to the global heap;  
};
```

Hoard's Blowup

Lemma. The maximum storage allocated in global heap is at most maximum storage allocated in local heaps.

Theorem. Let M be the user footprint for a program, and let H be Hoard's allocator footprint. We have

$$H \leq O(SP + M) ,$$

and hence the blowup is

$$H/M = O(SP/M + 1) . \blacksquare$$

Proof. Analyze the storage in local heaps.

Recall that $m_i \geq \min(h_i - 2S, h_i/2)$.

First term: at most $2S$ unutilized storage per heap for a total of $O(SP)$.

Second term: allocated storage is at most twice the used storage for a total of $O(M)$. $\blacksquare$

Other Solutions

jemalloc is like Hoard, with a few differences:

- **jemalloc** has a separate global lock for each different allocation size.
- **jemalloc** allocates the object with the smallest address among all objects of the requested size.
- **jemalloc** releases empty pages using
`madvise(p, MADV_DONTNEED, ...)` ,
which zeros the page while keeping the virtual address valid.
- **jemalloc** is a popular choice for parallel systems due to its performance and robustness.

SuperMalloc (see reading) is an interesting contender.

Allocator Speeds

Allocator	SLOC	32 threads
Default	6,281	0.9 M/s
Hoard	16,948	17.1 M/s
jemalloc	22,230	38.2 M/s
SuperMalloc	3,571	131.7 M/s

Summary

- Virtual Memory System: a layer of indirection
- **Stack**: basic form, efficient, but inflexible
- **Heap** is more general
 - Fixed-size allocation using **free lists**
 - Variable size allocation using **binned free lists**
 - Parallel heap allocation using **global + local heaps**
- Internal and external **fragmentation**